



AN ACADEMIC GRADING PLATFORM FOR BUNAWAN APLAYA ELEMENTARY SCHOOL

A Capstone Paper
Presented to the ITE Department
Philippine College of Technology
Gahol Drive, Davao City

In Fulfillment of the
Requirements for the course
Capstone 2

Members:

John Dave Bansag
Don Joshua Bongabong
Edmond Ambulario

June 2026

Abstract

The problem encountered by the Bunawan Aplaya Elementary school is that they lack an efficient system to keep track of their students' records since most of their processes are still being done manually. Parents also have no idea nor access of what their children are accomplishing in school since they do not have immediate contact with their children's teachers.

The Academic Grading Platform aims to serve as an easy-to-use tool for teachers and parents to communicate students' academic accomplishments. Built with HTML, CSS, JavaScript, Node.js, Express.js as technologies. The methodologies used in this development are the Waterfall model since the required activities to build the system are already defined through interviews and actually observing the school and its processes. Administrators, Teachers, and Students/Parents have different dashboards and they can only access parts of the website that are intended for them. Teachers can post grades by simply uploading an Excel file with their student's grades. This ensures that students will receive the correct grades from their teachers. Posted grades are subject for approval by the administrator. This grading platform consists of a login module, activate account module, student grades upload and management module, admin dashboard for easier view of student records, and a reports module. The system has been tested on different browsers as well as on different devices such as mobile phones to ensure that students can access it anywhere at anytime.

The Bunawan Aplaya Elementary School grading platform is hosted through Vercel and Railway which means that no software installation is required for this system to work. This system will benefit the school and students' parents by giving parents access of what their children are achieving in school. It can also help parents take part in improving their children's education as soon as they see signs of decline in grades.

Acknowledgment

This project is made possible through the help and support from everyone, including: parents, teachers, friends, and in essence all sentient beings. Especially, please allow us to dedicate our acknowledgment of gratitude toward the following significant adviser and contributors.

First and foremost, we would like to thank God for his unconditional guidance and wisdom as we make our project.

Second, we would like to thank our instructor MR. John Ray Josol and our coordinator Mrs. Marissa Antig for their most support and encouragement for giving us this project. This give us the experience on how to cooperate and engage ourselves in a serious project.

Finally, we sincerely thank to our parents, family, and friends who provide the advice and financial support. The product of this project would not be possible without all of them.

TABLE OF CONTENTS

Abstract	i
Acknowledgment Letter.....	ii
Chapter 1: Introduction.....	1
1.0 Project Context.....	1
1.1 Purpose and Description	1
1.3 Objectives	1
1.3.1 Specific objectives	2
1.4 Scope and limitation	2
1.4.1 Scope	2
1.4.2 Limitation	3
Chapter 2: Review of Related Literature / System	3
2.1 Local	3
2.2 National	3
2.3 International	4
Chapter 3: Methodology.....	4
3.1 Research Design.....	4
3.2 Development Tools.....	5
3.3 System Architecture.....	6
3.4 Data Flow Diagram.....	7
3.5 Entity Relationship Diagram.....	7
Chapter 4: Results and Discussion.....	9
4.1 System Features.....	9
4.2 System Interface.....	10
4.3 Testing and Evaluation.....	10
Chapter 5: Summary, Conclusion, and Recommendation.....	12
5.1 Summary.....	12
5.2 Conclusion.....	12
5.3 Recommendations.....	12
References.....	12
Appendices.....	14

Chapter I: Introduction

A **grade portal** is an online platform most commonly used in college universities and private institutions that would allow students and their parents to view their academic performances mainly focusing on grades and other school related information.

The researchers plan to develop a system with the title “An Academic Grading Platform for Bunawan Aplaya Elementary School” and as the title suggests, it would focus on a public elementary school far from the heart of the city where not much of modern technology is being used.

1.1 Project Context

Elementary schools still face many challenges today such as effectively monitoring student performances as a result of continuous use of managing records using outdated systems. On the parents' side, is the lack of access to information regarding the student's academic advancement. The lack of timely exchanges of

information on the performance of students from teachers to parents can have a negative impact on the academic performance of the student.

Integrating an accessible grading system will significantly help in monitoring the academic performance of the student for both teachers and parents. It will provide teachers the ability to input grades and document evaluations of the student's behavior toward specific subjects and activities. Different levels of access are implemented to control the actions that can be made, depending on the user being a teacher or parent, to ensure the integrity and privacy of data.

1.2 Purpose and Description

The grade portal is developed to improve the monitoring of the academic development of all students in Bunawan Aplaya Elementary School. It functions as a digital record for teachers to input the performance of the students while providing viewership for parents on the records to ensure early intervention depending on the

performance of the student.

The system is a web-based application that operates as an interactive digital platform to disseminate the academic performance of the students in Bunawan Aplaya Elementary provided by teachers for the parents to have up-to-date information on the progress of the academic development of the students.

1.3 Objectives

The overall goal of this research is to develop a grade portal for Bunawan Aplaya Elementary School to increase the efficiency of academic performance provided by teachers so the parents have access to the academic records of the student. To elaborate, this research aims to:

Develop a grading system: to increase the efficiency of academic performance monitoring of teachers and parents regarding the students.

Acts as a digital record: for teachers to upload and manage the grades of students efficiently to

improve work productivity.

Provide parents with real-time: information on the academic performance of the student.

Evaluates student performance: in the event of an increase or decrease in performance levels of the student.

1.4 Scope and Limitation

The scope of the study includes the development of a system that will assist teachers in recording and monitoring the grades of students, which will then be provided for parents to have access to view the academic progress of the student.

Scope

Teachers can upload the grades of students on the system for parents to view on the respective account of each student.

The functionality of the student dashboard is only focused on viewing grades as provided by the teacher.

Limitation

The website can only be accessed if the user is connected to the internet

The proposed system is limited to individuals with existing records in the school database; therefore, only authorized and registered users can use the system.

The system will only act as support for parents and teachers to monitor the academic performance of the student to uphold the traditional practices between parents and teachers.

Chapter II: Review of Related Literature

2.1 Local

Mobile-Web Responsive
Portal for St. John Paul II
College of Davao Applying
Predictive Analytics
Techniques

Studies have shown that the use of a Mobile-Web responsive portal in St. John Paul II College of Davao have named Classification and Regression

Tree (CART), the system addressed the drawbacks of traditional, manual systems that lead to increased productivity and user satisfaction given by the students and staff.

It is evident that with the use of effective online systems the users are satisfied with the fast paced services provided by the system, with this in mind the grade portal that the researchers are developing for Bunawan Aplaya Elementary School will help parents have quick access to the academic records of their children via mobile-web based portal.

National

Enhancing Parental
Engagement via the Grade
Viewer Application: A Study in
Taganaan National High School

As research suggests, parent involvement is crucial for the academic growth of a student, with the help of a grade viewing web application, it bridges the gap between the parents and school and its educators (Perfecto R. Ruaya, Jr., 2023).

Children need guidance in the early stages of their lives to prevent them from taking the wrong path. More often than not, children tend to make fewer mistakes when they know they are being watched.

With the use of an online website application such as the Grade Portal for Bunawan Aplaya Elementary School that enables parents to view the grades of the student, they can efficiently monitor the academic performance of the student whenever and wherever, as long as it is provided to them by the teacher via the online website.

International

Using School Websites for Home-School Communication and Parental Involvement

The research has found that schools often limit themselves to communicating only the social aspects of student development rather than issues that are related to teaching, which suggests that schools are indirectly encouraging parents

to focus on the overall well-being of the students, leaving the pedagogical matters to the teachers.

We, the researchers, have added features to the online website to encourage the school to address their concerns regarding parental volunteerism in school activities to enhance home-school relationships for more productive results in the enhancement of the academic performance of the students.

CHAPTER III: TECHNICAL BACKGROUND AND METHODOLOGY

3.1 Research Design

This study uses a developmental research design using the Waterfall methodology as the systematic framework for developing the Academic Grading Platform for Bunawan Aplaya Elementary School. The use of developmental research is appropriate for this study because the main objective is to design and build a functional information system that addresses a problem that has always been present in

underdeveloped schools with regards to managing and

The Waterfall methodology was selected as the development framework because the requirements for the grading platform have already been laid out. The core functionalities of the system were developed to cater issues that were identified at the outset through interviews and observations conducted at the school. Since the circumstances are unlikely to shift significantly during the development period, the sequential structure of Waterfall provides a structured process from planning to deployment.

3.2 Development Tools

The researchers used different software and technologies in developing the Academic Grading Platform for Bunawan Aplaya Elementary School. These tools helped in designing, coding, testing, and deploying the system efficiently.

Frontend Technologies:

These are the tools used to create

what the users see and interact with, they provide its appeal and interactions for a user-friendly experience.

The frontend of the system was developed using:

HTML – acts as the backbone of our system in which it sets the structure of the for each header, button, or image.

CSS – gives appeal to the system providing aspects that is not hard for the user's eyes.

JavaScript – used for interactive functions and dynamic features of the website.

Backend Technologies:

To ensure system internal functionality, these tools were used to apply logic to the system, handle user requests, and manage data processing.

The backend of the system was developed using:

Node.js – used as the server-side environment to process request and handle system operations.

Express.js – used to build APIs, routes, authentication, and backend functions.

Database Management:

Tools used to store data of user to ensure accurate information are provided to users.

The data of the system was stored using:

MySQL — used as the main database for storing student records, grades, school years, sections, and subjects.

MySQL Workbench — used to design, manage, and execute database queries.

Development Tools:

The researchers utilized the following tools:

Visual Studio Code – is the development environment used to build the system, here is where the source code was written, where backend logic was implemented, and develop internal functionalities.

Thunder Client — used for API testing and checking backend requests.

File and Data Processing Libraries:

Specific tools used to integrate external data files provided by user to upload to the system.

External user uploads were accepted using:

Multer — used for file upload handling.

SheetJS — used for reading and processing Excel files for grade uploads.

3.3 System Architecture

The system follows a client-server architecture.

Presentation Layer: it is the interface users can see and interact with, which was built using HTML, CSS, and JavaScript.

Application Layer: To implement logic to the system Node.js and Express.js have been used. It handles user requests, build user authentication, APIs routes, and backend functions.

Data Layer: MYSQL was used to store user data for its high performance in handling data with

integrity and compatibility with other existing tools that have been used to develop the system.



Figure 3.3.1: System Architecture Diagram

3.4 Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how information moves within the system, it shows how data is collected and distributed to ensure functionality and user authentication is implemented for user security.

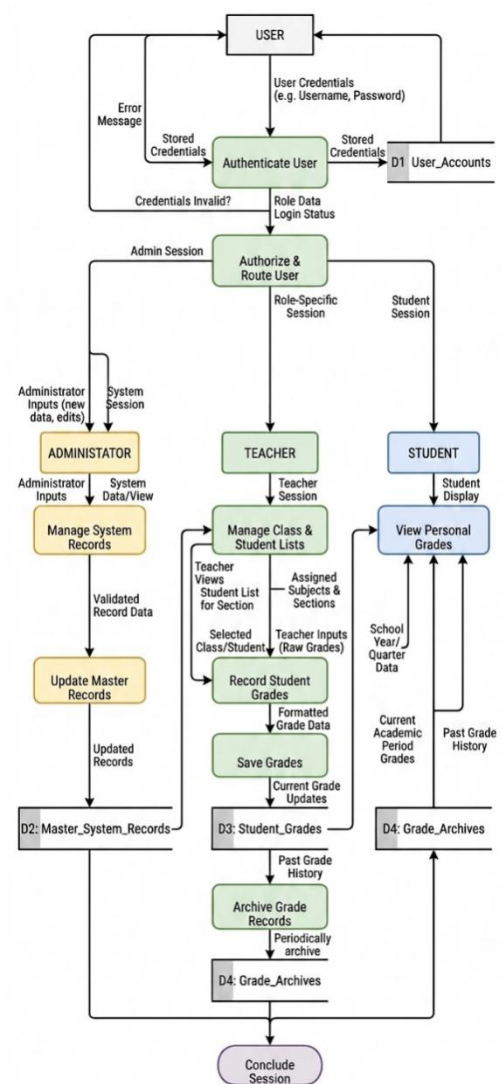


Figure 3.4.1: Data Flow Diagram

3.5 Entity Relationship Diagram

The Grading Portal System database is composed of eleven interrelated tables that collectively manage user authentication, academic records, and grade submission workflows. The database is designed to support three types of users: administrators, teachers, and students, each with distinct roles and access levels

within the system. The following sections describe each entity (table) in the database, including its attributes, data types, and relationships with other entities.

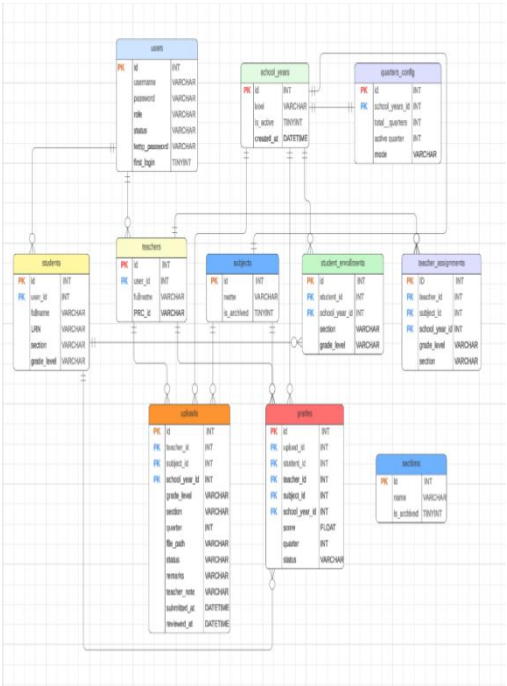


Figure 3.5.1: Entity Relationship Diagram

The entities are as follows:

Users

Column Name	Data Type	Key	Description
id	INT	PK	Unique identifier for each user
username	VARCHAR		Login username of the user
password	VARCHAR		Hashed password for authentication
role	VARCHAR		Role: admin, teacher, or student
status	VARCHAR		Account status: activated or not activated
temp_password	VARCHAR		Temporary password for first-time login
first_login	TINYINT		Flag indicating if user has logged in before

Table 3.5.1: User Table

Students

Column Name	Data Type	Key	Description
id	INT	PK	Unique identifier for each student
user_id	INT	FK → users	References the user account of the student
full_name	VARCHAR		Full name of the student
lrn	VARCHAR		Learner Reference Number (LRN)
section	VARCHAR		Current section of the student
grade_level	VARCHAR		Current grade level of the student

3.5.2: Students Table

Teachers

Column Name	Data Type	Key	Description
id	INT	PK	Unique identifier for each teacher
user_id	INT	FK → users	References the user account of the teacher
full_name	VARCHAR		Full name of the teacher
prc_id	VARCHAR		PRC ID number of the teacher

3.5.3: Teachers table

School years

Column Name	Data Type	Key	Description
id	INT	PK	Unique identifier for the school year
label	VARCHAR		School year label (e.g., 2024-2025)
is_active	TINYINT		Flag indicating the currently active school year
created_at	DATETIME		Date and time the school year was created

3.5.3: School Years Table

Quarters Configuration

Column Name	Data Type	Key	Description
id	INT	PK	Unique identifier for the configuration
school_year_id	INT	FK → school_years	References the associated school year
total_quarters	INT		Total number of quarters in the school year
active_quarter	INT		Currently active quarter number
mode	VARCHAR		Quarter mode: manual or automatic

3.5.3: Quarters Configuration Table

Subjects

Column Name	Data Type	Key	Description
id	INT	PK	Unique identifier for each subject
name	VARCHAR		Name of the subject
is_archived	TINYINT		Flag indicating if subject is archived

3.5.3: Subjects Table

Sections

Column Name	Data Type	Key	Description
id	INT	PK	Unique identifier for each section
name	VARCHAR		Name of the section
is_archived	TINYINT		Flag indicating if section is archived

3.5.3: Sections Table

Uploads

Column Name	Data Type	Key	Description
id	INT	PK	Unique identifier for each upload
teacher_id	INT	FK → teachers	References the teacher who submitted the upload
subject_id	INT	FK → subjects	References the subject of the upload
school_year_id	INT	FK → school_years	References the school year of the upload
grade_level	VARCHAR		Grade level for this upload
section	VARCHAR		Section for this upload
quarter	INT		Quarter number for this upload
file_path	VARCHAR		File path of the uploaded Excel file
status	VARCHAR		Status: pending, approved, rejected, or locked
remarks	VARCHAR		Admin remarks for rejected uploads
teacher_note	VARCHAR		Optional note from the teacher
submitted_at	DATETIME		Date and time of submission
reviewed_at	DATETIME		Date and time of admin review

3.5.3: Uploads Table

CHAPTER IV: Results and Discussion

4.1 System Features

The Grade Portal introduces features that meet user needs and satisfaction while ensuring digital inclusivity to all users through accessible means and promoting a user-friendly environment.

Login Module: The user login provides access to admin, student, and teachers and uses unique identifiers that primarily exist on school records to ensure the security and integrity of our system to prevent outside entities in taking hold of confidential information.

Student Profile Module:

Registered students have access to their user profile after being successfully registered and activating the account through user authentication backed up system data, that lets them view their grade for each grading period through web browsers.

Teacher Profile Module: Teachers are able to upload grades using excel files from excel files from their existing records for students to view, they will also have lists of classes they supervise shown on record.

Admin Dashboard Module: User registration can only be made through the admin dashboard to ensure only school registered users get accounts made on their behalf to ensure user security and information safety will not be compromised.

Reports Generation: This feature tracks all existing accounts on the system, enable Admin users to tally the number of records from various categories for simpler data handling.

4.2 System Interface

The system interface was made from a user's perspective and satisfaction to prevent any confusions or user errors. By using straightforward symbols and descriptions, users can navigate the system seamlessly.

Admin Dashboard

Shown in Figure 4.2.1 is the Admin Dashboard Homepage, presenting the tally of all accounts being handled, a tracker or actions that have been made, and pending tasks with a simple yet comprehensive design.

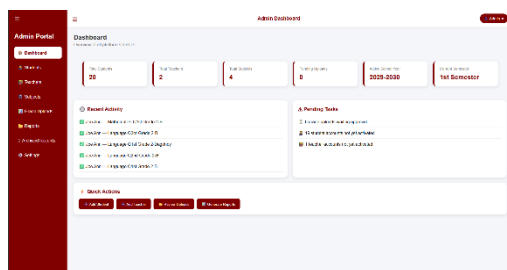


Figure 4.2.1: Admin Dashboard Interface

Teacher Dashboard

In Figure 4.2.2 shows the Teacher Dashboard that presents their assigned subjects and classes, including shortcuts of actions they can do to improve productivity.

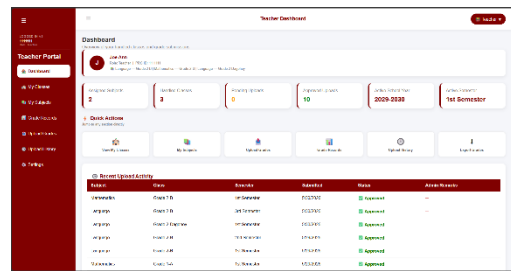


Figure 4.2.2: Teacher Dashboard Interface

Student Dashboard

Figure 4.2.3 shows the Student Dashboard presenting the current grade level of the student, which class, and the recent grades that have been recently uploaded.

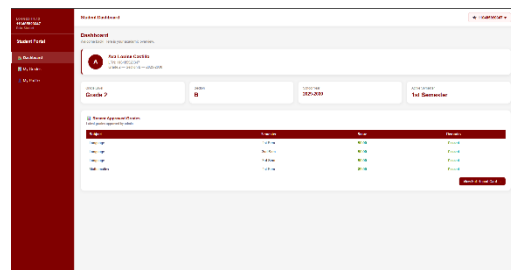


Figure 4.2.3: Student Dashboard Interface

4.3 Testing and Evaluation

The Grading Portal System is a web-based application developed to digitize and streamline the grade management process. The system was developed and tested on multiple hardware devices and is accessible through modern web browsers on both desktop and mobile platforms. This section outlines the hardware and software specifications used during development, as well as the

minimum requirements for end users to access the system.

4.3.1 Compatibility Testing

The system was tested across multiple devices, browsers, and operating systems to ensure broad accessibility and compatibility. The following table summarizes the devices and browsers used during compatibility testing:

Desktop and Laptop Browsers: shown in table 4.3.1 are compatible browsers for Grade Portal System.

Browser	Version	Result
Google Chrome	Latest	✓ Compatible
Microsoft Edge	Latest	✓ Compatible
Opera GX	Latest	✓ Compatible

Table 4.3.1: Desktop and Browser Compatibility Testing

Mobile Devices: shown in table are compatible mobile devices for the Grade Portal System.

Brand	OS	Result
Samsung	Android	✓ Compatible
Infinix	Android	✓ Compatible
Vivo	Android	✓ Compatible
OPPO	Android	✓ Compatible
Xiaomi	Android	✓ Compatible
POCO	Android	✓ Compatible

Table 4.3.2: Mobile Devices Compatibility Testing

4.3.2 Minimum System Requirements for End Users

Since the Grading Portal System is a web-based application hosted on cloud servers (Vercel for frontend and Railway for backend), end users are not required to install any software. The following are the minimum requirements to access and use the system:

Desktop and Laptop: shown in table 4.3.3 are the minimum requirements for desktops and laptops users can use to access the Grade Portal System.

Component	Specification
Processor	Intel Celeron / AMD equivalent or higher
RAM	2GB or higher
Storage	No local storage required (web-based)
Operating System	Windows 7 or later / macOS / Linux
Browser	Google Chrome, Microsoft Edge, Firefox, Opera (latest versions)
Internet Connection	Stable broadband or mobile data connection
Screen Resolution	1024 x 768 minimum (1366 x 768 recommended)

Table 4.3.3: Minimum System Requirement for End Users for Desktop and Laptop

Mobile Devices: shown in table 4.3.4 are the minimum requirements for mobile devices users can use to access the Grade Portal System.

Component	Specification
Device Type	Android smartphone or tablet
Android Version	Android 8.0 (Oreo) or higher
RAM	2GB or higher
Browser	Google Chrome or any modern mobile browser
Internet Connection	Mobile data (3G/4G/5G) or Wi-Fi
Screen Size	5 inches or larger recommended

Table 4.3.4: Minimum System Requirement for End Users for Mobile Devices

Chapter V: Summary, Conclusion, and Recommendations

5.1 Summary

The Academic Grading Portal for Bunawan Aplaya Elementary school is an online platform that is designed to allow students and parents to have real-time access to student performance, at the same time providing an effective data management tool for school administration and faculty.

5.2 Conclusion

The Academic Grading Portal for Bunawan Aplaya Elementary School will help the parents monitor their child's performance to ensure they perform well in school to establish a solid foundation of their education at a young age.

5.3 Recommendations

Bunawan Aplaya Grading Portal Application Development

To enhance user accessibility and productivity, a mobile application counterpart of the website should be developed to provide easier access.

An Embedded Mini-game for student Learning and Interactivity

To encourage students in engaging with the website, an interactive game mini-game will be integrated in the future. This feature would monitor the progress of the students and be given added points by teachers after reaching a milestone achievement.

SMS Offline Notification Feature

To improve productivity of the users, an offline notification feature via SMS should be added to alert users of tasks, uploads, and announcements for inclusivity particularly to user who has limited internet connectivity.

References

- Adorico, K. S. O., & Yara, F. (2024). Mobile-Web Responsive Portal for St. John Paul II College of Davao Applying Predictive Analytics Techniques.
https://www.researchgate.net/publication/381229033_Mobile-Web_Responsive_Portal_for_St_John_Paul_II_College_of_Davao_Appling_Predictive_Analytics_Techniques
- Ruaya, P. R., Jr. (2023). Enhancing parental engagement via the Grade Viewer Application: A study in

Taganaan National High School.
*International Journal of Advanced
Research in Science,
Communication and Technology*,
3(2), 177–183.
https://www.researchgate.net/publication/372756475_Enhancing_Parental_Engagement_via_the_Grade_Viewer_Application_A_Study_in_Taganaan_National_High_School

Gu, L. (2017). Using school websites for home–school communication and parental involvement? *Nordic Journal of Studies in Educational Policy*, 3(2), 133–143.

<https://www.tandfonline.com/doi/full/10.1080/20020317.2017.1338498>

APPENDICIES

- A. Request Letter**
- B. Acceptance Letter**
- C. Certificate of Deployment**
- D. Grammarian Certificate**
- E. Organizational Structure**
 - a. Curriculum Vitae of Members**
- F. Ai Detector Result**
- G. Source Code**

REQUEST LETTER

April 01, 2026

Chona F. Nieves, MAEM
DepEd Teacher III
Bunawan Aplaya Elementary School
Km. 24 Bunawan, Davao City

Dear Ma'am Nieves,

We, **John Davao Bansag, Don Joshua Bongabong, Edmond Ambulario**, Diploma in Information Technology students, respectfully request permission to conduct our Capstone study entitled “**An Academic Grading Platform for Bunawan Aplaya Elementary School.**”

In this regard, we seek approval to conduct our research at **Bunawan Aplaya Elementary School**, including interviews, data gathering, and system-related activities.

The study aims to develop a platform for teachers to upload the grades of their students for them to view online.

Rest assured that all information gathered will be kept confidential and used strictly for academic purposes.

We are looking forward for your favourable response to our request. Thank you in anticipation .

Respectfully yours,

The Researchers,

John Dave Bansag

Don Joshua Bongabong

Edmond Ambulario

Jeffrey D. Oñas, MIM
Program Head

Marissa Antig
Program Coordinator

John Ray Josol
Capstone Instructor

Chona F. Nieves, MAEM
DepEd Teacher III

ACCEPTANCE LETTER

May 10, 2026

Chona F. Nieves, MAEM
DepEd Teacher III
Bunawan Aplaya Elementary School
Km. 24 Bunawan, Davao City

Dear Ma'am Nieves,

This letter serves as formal approval for the capstone project entitled “**An academic Grading Platform for Bunawan Aplaya Elementary School.**”

After reviewing the project proposal, Bunawan Aplaya Elementary School, grants permission for the conduct, implementation, and pilot testing of the system within the institution. The project is recognized as a valuable initiative to enhance the documentation and monitoring of students' learning progress.

Approval is granted provided that all activities are coordinated with the administration, institutional policies are observed, collected data remain confidential and used solely for academic purposes, and regular school operations are not disrupted. A copy of the final system and documentation shall be submitted upon project completion.

We look forward to supporting the successful completion of this project.

Sincerely yours,

The Researchers,

John Dave Bansag

Don Joshua Bongabong

Edmond Ambulario

Jeffrey D. Oñas, MIM
Program Head

Marissa Antig
Program Coordinator

John Ray Josol
Capstone Instructor

Chona F. Nieves, MAEM
DepEd Teacher III

CERTIFICATION

May 18, 2026

Chona F. Nieves, MAEM
DepEd Teacher III
Bunawan Aplaya Elementary School
Km. 24 Bunawan, Davao City

Dear Ma'am Nieves,

We, the undersigned Diploma in Information Technology students, hereby certify that our capstone project entitled **"An academic Grading Platform for Bunawan Aplaya Elementary School"** has been officially approved, implemented, and deployed at **Bunawan Aplaya Elementary School**.

The system was developed as part of our academic requirements and has undergone planning, development, testing, and evaluation within the institution. It presents as a web-based school portal accessible via web browser on both desktop and mobile devices, that lets teachers upload the grades of their students online for both students and parents to review made specifically for administrators, teachers, and parents of Bunawan Aplaya Elementary School.

We were granted permission to conduct all necessary activities, including requirements gathering, system development, user testing, and deployment, in accordance with institutional policies.

This certification is issued upon the request of the above-named students for whatever legal and academic purposes it may serve.

Sincerely yours,

The Researchers,

John Dave Bansag

Don Joshua Bongabong

Edmond Ambulario

Jeffrey D. Oñas, MIM
Program Head

Marissa Antig
Program Coordinator

John Ray Josol
Capstone Instructor

Chona F. Nieves, MAEM
DepEd Teacher III

GRAMMARIAN CERIFICATE

GRAMMARIAN'S CERTIFICATE

This is to certify that the research manuscript entitled: **"An Academic Grading Platform for Bunawan Aplaya Elementary School"** has been carefully reviewed, edited, and proofread to ensure compliance with the standard rules of English grammar and academic writing. The manuscript underwent a comprehensive review for grammatical accuracy, sentence construction, spelling, punctuation, coherence, clarity, diction, and formatting consistency.

The study was evaluated to improve the overall organization of ideas, readability, and the paper's technical presentation. Necessary revisions and language enhancements were conducted to ensure that the content reflects proper academic tone and professional writing standards suitable for research documentation and presentation. Furthermore, the manuscript was subjected to grammar and originality assessment using available writing evaluation tools.

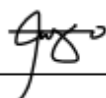
This certification is hereby issued upon the request of the researcher for academic compliance, documentation, and other lawful purposes it may serve.

Certified by:

Danica A. Requillo, LPT, MAELT

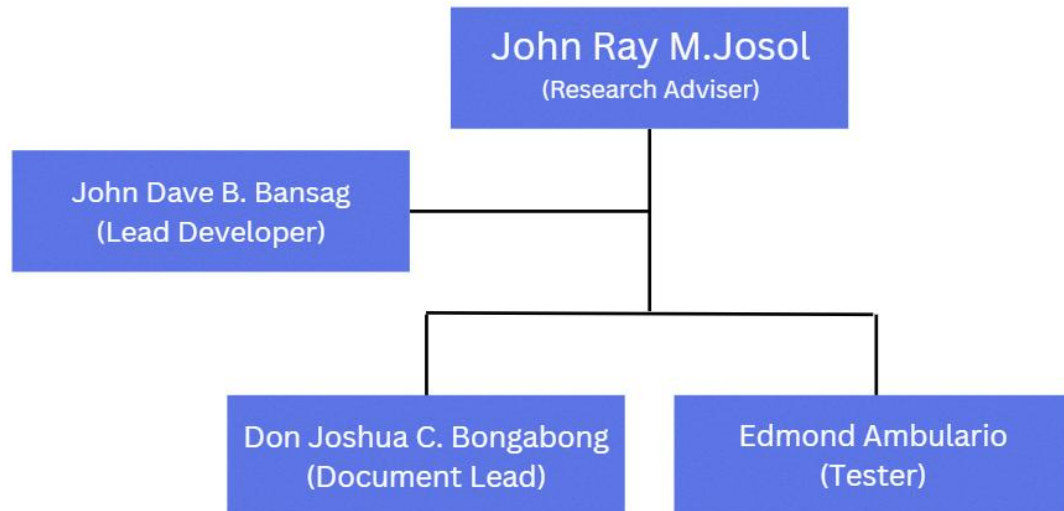
Grammarian

Signature: _____



Date: June 11, 2026

ORGANIZATIONAL STRUCTURE



Curriculum Vitae of Members

BANSAG, JOHN DAVE B.

+639311161546

BansagJohnDave915@gmail.com

Purok 7, Brgy. Mudiang, Bunawan District,
Davao City



EDUCATION

Primary
Mudiang Elementary School
2011-2017

Secondary (Junior High)
Mudiang High School
2017 - 2021

Secondary (Senior High)
Colegio de San Ignacio Inc.
2021 - 2023

College
Philippine College of Technology Inc.
2023-present

SKILLS

- Graphic Designing
- Web Designing
- Microsoft Tools Proficient
- Adaptability
- Basic Programming

LANGUAGES

- ENGLISH
- TAGALOG
- BISAYA

OBJECTIVE

I am seeking a role in a stimulating environment that offers challenges, fostering my personal and professional development. I aim to be part of a team where I can continually learn and grow.

PERSONAL INFORMATION

Date of Birth	:	September 15, 2005
Place of Birth	:	Agdao, Davao City
Gender	:	Male
Civil Status	:	Single
Citizenship	:	Filipino
Religion	:	Catholic
Height	:	5'7"
Weight	:	98kg
Father's Name	:	Mr. Cris Bansag
Mother's Name	:	Mrs. Daisy B. Bansag

QUALIFICATIONS/TRAININGS

VISUAL GRAPHIC DESIGN NCIII Passer

2D NCIII Passer

GRAPHIC DESIGNER

JKC

Bolton Street, Davao City

Technician Support

ARDCOM Electronics

Web/System Developer

Bunawan Aplaya Elementary School



Don Joshua Bongabong

CONTACT

Phone: (+63) 9097-806-712

Email: donjuswa@gmail.com

Address: Prk.4-A, Bunawan, Davao City

PROFESSIONAL EXPERIENCE

Production Staff | 2024

JKC Computer Services

- Assisted in crafting custom orders of clients based on their specifications, often in bulk orders
- Ensured timely and accurate completion of all customer orders under tight deadlines
- Developed attention to detail and improved time management during on-the-job training.

IT Support | 2025

Ardcom Electronic Parts

- Worked closely with a team to assemble bulk computer orders accurately and efficiently
- Assisted walk-in customers, answered product-related questions, and provided friendly service
- Maintained cleanliness and organization of the store to ensure a pleasant work environment

Customer Service Representative | 2025 – Present

Concentrix

- Efficiently resolve diverse customer inquiries through inbound and outbound calls, live chat, and email channels, consistently meeting quality assurance metrics and service level agreements
- Manage and monitor an ongoing case queue from opening to resolution, accurately documenting interactions and handling backend administrative tasks to ensure workflow efficiency.
- Work closely within a collaborative team environment to share best practices, optimize team performance, and support daily operational goals.

QUALIFICATIONS

Visual Grapgics Design NCIII Passer

2D Animation NCIII Passer

EDUCATION

Philippine College of Technology Inc | 2023–2027

Diploma in Information Technology

EDMOND D. AMBULARIO



CONTACT

📞 09454031446

✉️ edmondambulario@gmail.com

📍 Batangueno Vill., Matina
Aplaya, Davao City

EDUCATION

Philippine College Of Technology
Dimploma Information Technology
2023-2026

AWARDS & CERTIFICATIONS

- NCIII Visual Graphic Design
- NCIII 3D Animation

PROJECTS

My Digital Portfolio

Objective:

- To create a personal website that showcases my skills, projects, education, and achievements, providing an online presence for academic, professional, or personal purposes.

An Academic Grading Platform For Bunawan Aplaya Elementary School.

Objective:

- The objective of this project is to develop an online grade portal specifically for Bunawan Aplaya Elementary School. The portal aims to provide a centralized, efficient, and user-friendly platform for recording, tracking, and managing students' grades. It seeks to improve communication between teachers, students, and parents, reduce manual errors, and make it easier to monitor academic performance in real time.

SKILLS

- Languages: HTML, CSS, JAVASCRIPT, JAVA, PYTHON
- Database: MySQL, PHP, XAMPP
- Assemble and Disassemble PC

AI DETECTOR RESULT

The screenshot displays the GPTZero AI Detector web application. The browser address bar shows the URL `app.gptzero.me/source=landing`. The document being scanned is titled "Checked-An-Academic-Grading-Platform-for-Bunawan-Aplaya-Elementary-School-1.docx".

Document Content:

An Academic Grading Platform for Bunawan Aplaya Elementary School
A Capstone Project Presented to the Faculty
Of the Philippine College of Technology
In Partial Fulfillment of the Diploma in Information Technology

Members:
John Dave Bansag
Don Joshua Bongabong
Edmond Ambulario
June 2026
June 2, 2025
CHONA NIEVESAdviser

Dear Ma'am,
The undersigned, a 3rd year Diploma in Information Technology from the Philippine College of Technology who is presently proposing a study entitled "An Academic Grading Platform for Bunawan Aplaya Elementary School".
In connection with this, we would like to request from your good office to allow us to conduct our study to Grade-5 Diamond. Rest assured that the data gathered will be strictly for study purposes only and will be kept without-most confidentially.
We are looking forward for your favorable action on this regard.
Very truly yours,
Edmon Ambulario
Don Joshua Bongabong
John Dave Bansag
Approved By: Mr. John Ray Josol Instructor
Noted By: Mrs. Marissa Antig Program Head
Conforme: Mrs. Chona Nieves Adviser

Acknowledgment
This project is made possible through the help and support from everyone, including: parents, teachers, friends, and in essence all sentient beings. Especially, please allow us to dedicate our acknowledgment of gratitude toward the following significant adviser and contributors.
First and foremost, we would like to thank God for his unconditional guidance and wisdom as we make our project.
Second, we would like to thank our instructor MR. John Ray Josol and our coordinator Mrs. Marissa

AI Detector Results:

- Basic Scan:** human (GPTZero AI Detection Model 4.88)
- Confidence:** We are highly confident this text is entirely human.
- Chance this entire text is...:** AI 0%, Mixed 0%, Human 100%
- Ways to improve this draft:** Improve clarity, Correct grammar.
- Writing feedback on your draft:** Structure, tone, and argument analysis (Try for Free).
- Guidance for Teachers and reviewers:**

Text up-to-date: 9,910 characters • 1,307 words

Re-scan

SOURCE CODE

Database Configuration

```
const mysql = require('mysql2');
const path = require('path');
require('dotenv').config({ path: path.resolve(__dirname, '../.env') });

const db = mysql.createConnection({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
});

db.connect((err) => {
  if (err) {
    console.error('Database connection failed:', err.message);
    return;
  }
  console.log('Database connected successfully');
});

module.exports = db;
```

Authentication Middlewar

```
const jwt = require('jsonwebtoken');

const verifyToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if(!token){
    return res.status(401).json({ message: 'No token provided' });
  }

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if(err){
      return res.status(401).json({ message: 'Invalid or expired token' });
    }
    req.user = user;
    next();
  });
};

const isAdmin = (req, res, next) => {
  if(req.user.role !== 'admin'){
    return res.status(403).json({ message: 'Admin access only' });
  }
  next();
};

const isTeacher = (req, res, next) => {
  if(req.user.role !== 'teacher'){
    return res.status(403).json({ message: 'Teacher access only' });
  }
  next();
};

const isStudent = (req, res, next) => {
  if(req.user.role !== 'student'){
    return res.status(403).json({ message: 'Student access only' });
  }
  next();
};

const isTeacherOrAdmin = (req, res, next) => {
  if(req.user.role !== 'teacher' && req.user.role !== 'admin'){
    return res.status(403).json({ message: 'Teacher or admin access required' });
  }
  next();
};

module.exports = { verifyToken, isAdmin, isTeacher, isStudent, isTeacherOrAdmin };
```

Main Server Entry Point

```
const express = require('express');
const cors = require('cors');
const path = require('path');
require('dotenv').config();

const db = require('./config/db');

const authRoutes      = require('./routes/authRoutes');
const studentRoutes    = require('./routes/studentRoutes');
const teacherRoutes    = require('./routes/teacherRoutes');
const subjectRoutes    = require('./routes/subjectRoutes');
const schoolYearRoutes = require('./routes/schoolYearRoutes');
const gradeRoutes      = require('./routes/gradeRoutes');
const sectionRoutes    = require('./routes/sectionRoutes');
const reportRoutes     = require('./routes/reportRoutes');
const app = express();

app.use((req, res, next) => {
  const origin = req.headers.origin;
  if (origin && (origin.includes('netlify.app') ||
origin.includes('vercel.app')))) {
    res.header("Access-Control-Allow-Origin", origin);
  }
  res.header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE,
OPTIONS");
  res.header("Access-Control-Allow-Headers", "Origin, X-Requested-With,
Content-Type, Accept, Authorization");
  res.header("Access-Control-Allow-Credentials", "true");
  if (req.method === "OPTIONS") {
    return res.status(200).end();
  }
  next();
});
app.use(express.json());
app.use(express.static(path.join(__dirname, '../')));
app.use('/assets', express.static(path.join(__dirname, '../assets')));
app.get('/health', (req, res) => res.status(200).send('OK'));
app.use('/api/auth',      authRoutes);
app.use('/api/students',  studentRoutes);
app.use('/api/teachers',  teacherRoutes);
```

```
app.use('/api/subjects',    subjectRoutes);
app.use('/api/school-years', schoolYearRoutes);
app.use('/api/grades',      gradeRoutes);
app.use('/api/sections',    sectionRoutes);
app.use('/api/reports',     reportRoutes);

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, '../index.html'));
});

process.on('uncaughtException', (err) => {
  console.error('Uncaught Exception:', err.message);
  console.error(err.stack);
});

const PORT = process.env.PORT || 8080;
app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server running on port ${PORT}`);
});
```

Authentication Controller

```
const db = require('../config/db');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
require('dotenv').config();

// =====
// LOGIN
// =====

const login = (req, res) => {

  const { username, password } = req.body;

  if(!username || !password){
    return res.status(400).json({
      message: 'Username and password are required'
    });
  }

  const sql = 'SELECT * FROM users WHERE username = ?';

  db.query(sql, [username], async (err, results) => {

    if(err) return res.status(500).json({ message: 'Server error' });

    if(results.length === 0){
      return res.status(401).json({
        message: 'Invalid username or password'
      });
    }

    const user = results[0];

    // block if not activated
    if(user.status === 'not_activated'){
      return res.status(403).json({
        message: 'Account not yet activated. Please activate your
account first.',
        code: 'NOT_ACTIVATED'
      });
    }

    // check password
    const isMatch = await bcrypt.compare(password, user.password);

    if(!isMatch){
```



```

        return res.status(401).json({
            message: 'Invalid username or password'
        });
    }

    // generate JWT
    const token = jwt.sign(
        { id: user.id, role: user.role },
        process.env.JWT_SECRET,
        { expiresIn: '24h' }
    );

    res.json({
        message: 'Login successful',
        token,
        role: user.role,
        username: user.username
    });

});

});

// =====
// ACTIVATE – STUDENT
// =====

const activateStudent = async (req, res) => {

    const {
        lrn,
        full_name,
        temp_password,
        new_password,
        confirm_password
    } = req.body;

    if(!lrn || !full_name || !temp_password || !new_password ||
    !confirm_password){
        return res.status(400).json({ message: 'All fields are required'
    });
    }

    if(new_password !== confirm_password){
        return res.status(400).json({ message: 'Passwords do not match' });
    }

    if(new_password.length < 8){

```

```

    return res.status(400).json({
      message: 'Password must be at least 8 characters'
    });
  }

  // find student by LRN
  const sql = `
    SELECT s.*, u.status, u.id AS user_id, u.temp_password AS
stored_temp
    FROM students s
    JOIN users u ON s.user_id = u.id
    WHERE s.lrn = ?
  `;

  db.query(sql, [lrn], async (err, results) => {

    if(err) return res.status(500).json({ message: 'Server error' });

    if(results.length === 0){
      return res.status(404).json({
        message: 'LRN not found. Please contact your administrator.'
      });
    }

    const student = results[0];

    // already activated?
    if(student.status === 'activated'){
      return res.status(400).json({
        message: 'Account already activated. Please login normally.',
        code: 'ALREADY_ACTIVATED'
      });
    }

    // verify full name
    const dbName = student.full_name.toLowerCase().trim();
    const inputName = full_name.toLowerCase().trim();

    if(dbName !== inputName){
      return res.status(401).json({
        message: 'Full name does not match our records.'
      });
    }

    // verify temp password against stored plain temp
    if(temp_password !== student.stored_temp){
      return res.status(401).json({
        message: 'Incorrect temporary password.'
      });
    }
  });
}

```

```

    });
  }

  // all verified – activate
  completeActivation(student.user_id, new_password, res);

});

};

// =====
// ACTIVATE – TEACHER
// =====

const activateTeacher = async (req, res) => {

  const {
    prc_id,
    full_name,
    temp_password,
    new_password,
    confirm_password
  } = req.body;

  if(!prc_id || !full_name || !temp_password || !new_password ||
!confirm_password){
    return res.status(400).json({ message: 'All fields are required'
});
  }

  if(new_password !== confirm_password){
    return res.status(400).json({ message: 'Passwords do not match' });
  }

  if(new_password.length < 8){
    return res.status(400).json({
      message: 'Password must be at least 8 characters'
    });
  }

  // find teacher by PRC ID
  const sql = `
    SELECT t.*, u.status, u.id AS user_id, u.temp_password AS
stored_temp
    FROM teachers t
    JOIN users u ON t.user_id = u.id
    WHERE t.prc_id = ?
  `;

```

```

db.query(sql, [prc_id], async (err, results) => {

  if(err) return res.status(500).json({ message: 'Server error' });

  if(results.length === 0){
    return res.status(404).json({
      message: 'PRC ID not found. Please contact your administrator.'
    });
  }

  const teacher = results[0];

  // already activated?
  if(teacher.status === 'activated'){
    return res.status(400).json({
      message: 'Account already activated. Please login normally.',
      code: 'ALREADY_ACTIVATED'
    });
  }

  // verify full name
  const dbName = teacher.full_name.toLowerCase().trim();
  const inputName = full_name.toLowerCase().trim();

  if(dbName !== inputName){
    return res.status(401).json({
      message: 'Full name does not match our records.'
    });
  }

  // verify temp password
  if(temp_password !== teacher.stored_temp){
    return res.status(401).json({
      message: 'Incorrect temporary password.'
    });
  }

  // all verified – activate
  completeActivation(teacher.user_id, new_password, res);

});
};
// =====
// COMPLETE ACTIVATION
// =====

async function completeActivation(userId, newPassword, res){

```

```

try {

  const hashedPassword = await bcrypt.hash(newPassword, 10);

  const sql = `
    UPDATE users
    SET password = ?,
        status = 'activated',
        temp_password = NULL,
        first_login = 0
    WHERE id = ?
  `;

  db.query(sql, [hashedPassword, userId], (err) => {
    if(err) return res.status(500).json({ message: 'Server error' });

    res.json({
      message: 'Account activated successfully! You can now login.'
    });
  });

} catch(err) {
  res.status(500).json({ message: 'Server error' });
}

}

const changeAdminCredentials = async (req, res) => {

  const { current_password, new_username, new_password } = req.body;
  const userId = req.user.id;

  if(!current_password){
    return res.status(400).json({
      message: 'Current password is required'
    });
  }

  if(!new_username && !new_password){
    return res.status(400).json({
      message: 'Provide a new username or new password'
    });
  }

  try {

    // get current user
    const [users] = await db.promise().query(

```

```

    'SELECT * FROM users WHERE id = ?',
    [userId]
  );

  if(users.length === 0){
    return res.status(404).json({ message: 'User not found' });
  }

  const user = users[0];

  // verify current password
  const isMatch = await bcrypt.compare(current_password,
  user.password);
  if(!isMatch){
    return res.status(400).json({
      message: 'Current password is incorrect'
    });
  }

  // build update
  const updates = [];
  const params = [];

  if(new_username && new_username.trim() !== user.username){
    // check username not taken
    const [existing] = await db.promise().query(
      'SELECT id FROM users WHERE username = ? AND id != ?',
      [new_username.trim(), userId]
    );
    if(existing.length > 0){
      return res.status(400).json({
        message: 'Username already taken'
      });
    }
    updates.push('username = ?');
    params.push(new_username.trim());
  }

  if(new_password){
    if(new_password.length < 6){
      return res.status(400).json({
        message: 'New password must be at least 6 characters'
      });
    }
    const hashed = await bcrypt.hash(new_password, 10);
    updates.push('password = ?');
    params.push(hashed);
  }

```

```

    if(updates.length === 0){
      return res.status(400).json({
        message: 'No changes detected'
      });
    }

    params.push(userId);

    await db.promise().query(
      `UPDATE users SET ${updates.join(', ')} WHERE id = ?`,
      params
    );

    res.json({
      message: 'Credentials updated successfully. Please log in again.',
      username_changed: !!new_username
    });

  } catch(err) {
    console.error('Change credentials error:', err);
    res.status(500).json({ message: 'Server error' });
  }
};

const changePassword = async (req, res) => {

  const { current_password, new_password } = req.body;
  const userId = req.user.id;

  if(!current_password || !new_password){
    return res.status(400).json({
      message: 'Current password and new password are required'
    });
  }

  if(new_password.length < 6){
    return res.status(400).json({
      message: 'New password must be at least 6 characters'
    });
  }

  try {

    const [users] = await db.promise().query(
      'SELECT * FROM users WHERE id = ?',
      [userId]
    );
  }
};

```

```

    );

    if(users.length === 0){
        return res.status(404).json({ message: 'User not found' });
    }

    const isMatch = await bcrypt.compare(current_password,
users[0].password);
    if(!isMatch){
        return res.status(400).json({ message: 'Current password is
incorrect' });
    }

    const hashed = await bcrypt.hash(new_password, 10);

    await db.promise().query(
        'UPDATE users SET password = ? WHERE id = ?',
        [hashed, userId]
    );
    res.json({
        message: 'Password changed successfully. Please log in again.'
    });

} catch(err) {
    console.error('Change password error:', err);
    res.status(500).json({ message: 'Server error' });
}
};

module.exports = {
    login,
    activateStudent,
    activateTeacher,
    changeAdminCredentials,
    changePassword
};

```


Student Controller (All functions)

```
const db      = require('../config/db');
const bcrypt  = require('bcryptjs');

function generateTempPassword(fullName, idNumber) {
  const nameParts    = fullName.trim().split(' ');
  const firstName     = nameParts[0];
  const lastName      = nameParts[nameParts.length - 1];
  const firstInitial  = firstName[0].toUpperCase();
  const last4         = String(idNumber).slice(-4);
  return `${firstInitial}.${lastName}${last4}`;
}

const getAllStudents = (req, res) => {

  const sql = `
    SELECT
      s.id,
      s.lrn,
      s.full_name,
      s.is_archived,
      COALESCE(u.status, 'not_activated') AS status,
      se.grade_level,
      se.section,
      se.adviser,
      sy.label AS school_year,
      sy.id    AS school_year_id
    FROM students s
    LEFT JOIN users u ON s.user_id = u.id
    LEFT JOIN student_enrollments se ON s.id = se.student_id
    LEFT JOIN school_years sy
      ON se.school_year_id = sy.id
      AND sy.is_active = 1
    WHERE s.is_archived = 0
      AND se.school_year_id = (
        SELECT id FROM school_years WHERE is_active = 1 LIMIT 1
      )
    ORDER BY se.grade_level, se.section, s.full_name ASC
  `;

  db.query(sql, (err, results) => {
    if(err) return res.status(500).json({
      message: 'Server error',
      error: err.message
    });
    res.json(results);
  });
};
```

```

};

// GET all ARCHIVED students
const getArchivedStudents = (req, res) => {

  const sql = `
    SELECT
      s.id,
      s.lrn,
      s.full_name,
      s.archived_at,
      s.archive_reason,
      COALESCE(u.status, 'not_activated') AS status,
      se.grade_level,
      se.section,
      sy.label AS school_year
    FROM students s
    LEFT JOIN users u ON s.user_id = u.id
    LEFT JOIN student_enrollments se
      ON se.student_id = s.id
      AND se.id = (
        SELECT id FROM student_enrollments
        WHERE student_id = s.id
        ORDER BY id DESC
        LIMIT 1
      )
    LEFT JOIN school_years sy ON se.school_year_id = sy.id
    WHERE s.is_archived = 1
    ORDER BY s.archived_at DESC
  `;

  db.query(sql, (err, results) => {
    if(err) return res.status(500).json({ message: 'Server error'
  });
    res.json(results);
  });

};

// GET single student
const getStudent = (req, res) => {
  const { id } = req.params;
  db.query(
    'SELECT * FROM students WHERE id = ?',
    [id],
    (err, results) => {
      if(err) return res.status(500).json({ message: 'Server
error' });
    }
  );
};

```

```

        if(results.length === 0) return res.status(404).json({
message: 'Student not found' });
        res.json(results[0]);
    }
    );
};

// GET my profile (for logged-in student)
const getMyProfile = (req, res) => {
    const userId = req.user.id;
    const sql = `
        SELECT
            s.lrn, s.full_name,
            u.status,
            se.grade_level, se.section, se.adviser,
            sy.label AS school_year,
            sy.id    AS school_year_id
        FROM students s
        JOIN users u ON s.user_id = u.id
        JOIN student_enrollments se ON s.id = se.student_id
        JOIN school_years sy ON se.school_year_id = sy.id
        WHERE s.user_id = ?
        ORDER BY sy.is_active DESC, sy.id DESC
        LIMIT 1
    `;
    db.query(sql, [userId], (err, results) => {
        if(err) return res.status(500).json({ message: 'Server error'
});
        if(results.length === 0){
            return res.status(404).json({
                message: 'Profile not found. Contact your administrator.'
            });
        }
        res.json(results[0]);
    });
};

// ADD student
const addStudent = async (req, res) => {
    const { lrn, full_name, grade_level, section, adviser } =
req.body;
    if(!lrn || !full_name || !grade_level || !section){
        return res.status(400).json({ message: 'LRN, full name, grade
level, and section are required' });
    }
    try {
        const tempPassword = generateTempPassword(full_name, lrn);
        const hashedTemp    = await bcrypt.hash(tempPassword, 10);

```

```

    const userSql = `
        INSERT INTO users (username, password, role, status,
temp_password, first_login)
        VALUES (?, ?, 'student', 'not_activated', ?, 1)
    `;
    db.query(userSql, [lrn, hashedTemp, tempPassword], (err,
userResult) => {
        if(err){
            if(err.code === 'ER_DUP_ENTRY') return
res.status(400).json({ message: 'LRN already exists' });
            return res.status(500).json({ message: 'Server error',
error: err.message });
        }
        const userId = userResult.insertId;
        db.query(
            'INSERT INTO students (lrn, full_name, user_id) VALUES
(?, ?, ?)',
            [lrn, full_name, userId],
            (err2, studentResult) => {
                if(err2) return res.status(500).json({ message: 'Server
error' });
                const studentId = studentResult.insertId;
                db.query('SELECT id FROM school_years WHERE is_active =
1 LIMIT 1', (err3, years) => {
                    if(err3 || years.length === 0){
                        return res.status(500).json({ message: 'No active
school year found' });
                    }
                    db.query(
                        'INSERT INTO student_enrollments (student_id,
school_year_id, grade_level, section, adviser) VALUES (?, ?, ?,
?, ?)',
                        [studentId, years[0].id, grade_level, section,
adviser || null],
                        (err4) => {
                            if(err4) return res.status(500).json({ message:
'Server error' });
                            res.status(201).json({
                                message: 'Student added successfully',
                                id: studentId,
                                temp_password: tempPassword
                            });
                        }
                    );
                });
            }
        );
    });

```

```

    } catch(err) {
      res.status(500).json({ message: 'Server error' });
    }
  };

// UPDATE student
const updateStudent = (req, res) => {
  const { id } = req.params;
  const { full_name, grade_level, section, adviser } = req.body;
  db.query('UPDATE students SET full_name = ? WHERE id = ?',
    [full_name, id], (err) => {
      if(err) return res.status(500).json({ message: 'Server error'
    });
      const sql = `
        UPDATE student_enrollments se
        JOIN school_years sy ON se.school_year_id = sy.id
        SET se.grade_level = ?, se.section = ?, se.adviser = ?
        WHERE se.student_id = ? AND sy.is_active = 1
      `;
      db.query(sql, [grade_level, section, adviser, id], (err2) =>
      {
        if(err2) return res.status(500).json({ message: 'Server
error' });
        res.json({ message: 'Student updated successfully' });
      });
    });
  };

const bulkArchiveStudents = (req, res) => {
  const { ids, reason } = req.body;
  if(!ids || !Array.isArray(ids) || ids.length === 0){
    return res.status(400).json({ message: 'No student IDs
provided' });
  }
  const sql = `
    UPDATE students
    SET is_archived      = 1,
        archived_at      = NOW(),
        archive_reason = ?
    WHERE id IN (?)
  `;
  db.query(sql, [reason || null, ids], (err, result) => {
    if(err) return res.status(500).json({ message: 'Server error'
  });
    res.json({
      message: `${result.affectedRows} student(s) archived
successfully`
    });
  });
};

```

```

    });
};

// BULK RESTORE students
const bulkRestoreStudents = (req, res) => {
  const { ids } = req.body;
  if(!ids || !Array.isArray(ids) || ids.length === 0){
    return res.status(400).json({ message: 'No student IDs
provided' });
  }
  const sql = `
    UPDATE students
    SET is_archived = 0, archived_at = NULL
    WHERE id IN (?)
  `;
  db.query(sql, [ids], (err, result) => {
    if(err) return res.status(500).json({ message: 'Server error'
});
    res.json({
      message: `${result.affectedRows} student(s) restored
successfully`
    });
  });
};

// BULK PERMANENT DELETE students
const bulkDeleteStudents = (req, res) => {
  const { ids } = req.body;
  if(!ids || !Array.isArray(ids) || ids.length === 0){
    return res.status(400).json({ message: 'No student IDs
provided' });
  }
  // get user_ids first
  db.query('SELECT user_id FROM students WHERE id IN (?)', [ids],
(err, results) => {
    if(err) return res.status(500).json({ message: 'Server error'
});
    const userIds = results.map(r => r.user_id).filter(Boolean);
    // delete enrollments
    db.query('DELETE FROM student_enrollments WHERE student_id IN
(?)', [ids], (err2) => {
      if(err2) return res.status(500).json({ message: 'Server
error' });
      // delete students
      db.query('DELETE FROM students WHERE id IN (?)', [ids],
(err3) => {
        if(err3) return res.status(500).json({ message: 'Server
error' });

```

```

        // delete user accounts if any
        if(userIds.length > 0){
            db.query('DELETE FROM users WHERE id IN (?)',
[userIds], (err4) => {
                if(err4) return res.status(500).json({ message:
'Server error' });
                res.json({ message: `${ids.length} student(s)
permanently deleted` });
            });
        } else {
            res.json({ message: `${ids.length} student(s)
permanently deleted` });
        }
    });
});
});
};

// legacy single archive (kept for compatibility)
const archiveStudent = (req, res) => {
    const { id } = req.params;
    db.query(
        'UPDATE students SET is_archived = 1, archived_at = NOW()
WHERE id = ?',
        [id],
        (err) => {
            if(err) return res.status(500).json({ message: 'Server
error' });
            res.json({ message: 'Student archived successfully' });
        }
    );
};

// legacy delete (kept for compatibility)
const deleteStudent = (req, res) => {
    res.status(400).json({
        message: 'Use bulk archive or bulk delete instead'
    });
};

const getStudentsByClass = (req, res) => {

    const { grade_level, section } = req.query;

    if(!grade_level || !section){
        return res.status(400).json({
            message: 'grade_level and section are required'
        });
    }
};

```

```

    }

    const sql = `
        SELECT
            s.id,
            s.lrn,
            s.full_name,
            se.grade_level,
            se.section
        FROM students s
        JOIN student_enrollments se ON s.id = se.student_id
        JOIN school_years sy ON se.school_year_id = sy.id
        WHERE se.grade_level = ?
            AND se.section = ?
            AND sy.is_active = 1
            AND s.is_archived = 0
        ORDER BY s.full_name ASC
    `;

    db.query(sql, [grade_level, section], (err, results) => {
        if(err) return res.status(500).json({
            message: 'Server error',
            error: err.message
        });
        res.json(results);
    });

};

module.exports = {
    getAllStudents,
    getArchivedStudents,
    getStudent,
    getMyProfile,
    getStudentsByClass,
    addStudent,
    updateStudent,
    deleteStudent,
    archiveStudent,
    bulkArchiveStudents,
    bulkRestoreStudents,
    bulkDeleteStudents
};

```


Grade Controller (All Functions)

```
const db = require('../config/db');
const multer = require('multer');
const xlsx = require('xlsx');
const path = require('path');
const fs = require('fs');

// =====
// MULTER STORAGE CONFIG
// saves uploaded excel files
// =====

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    const uploadPath = path.join(__dirname,
    '../..../uploads/excel-files');
    if(!fs.existsSync(uploadPath)){
      fs.mkdirSync(uploadPath, { recursive: true });
    }
    cb(null, uploadPath);
  },
  filename: (req, file, cb) => {
    const timestamp = Date.now();
    cb(null, `grades_${timestamp}_${file.originalname}`);
  }
});

const upload = multer({
  storage,
  fileFilter: (req, file, cb) => {
    const allowed = ['.xlsx', '.xls'];
    const ext = path.extname(file.originalname).toLowerCase();
    if(!allowed.includes(ext)){
      return cb(new Error('Only Excel files are allowed'));
    }
    cb(null, true);
  }
});

// =====
// GET ALL UPLOADS
// for admin - all uploads
// with teacher and subject info
// =====

const getAllUploads = (req, res) => {

  const sql = `
```

```

SELECT
    u.id,
    u.grade_level,
    u.section,
    u.quarter,
    u.status,
    u.remarks,
    u.teacher_note,
    u.submitted_at,
    u.reviewed_at,
    t.full_name AS teacher_name,
    t.prc_id,
    s.name AS subject_name,
    s.id AS subject_id,
    sy.label AS school_year,
    sy.id AS school_year_id
FROM uploads u
JOIN teachers t ON u.teacher_id = t.id
JOIN subjects s ON u.subject_id = s.id
JOIN school_years sy ON u.school_year_id = sy.id
WHERE sy.is_active = 1
ORDER BY u.submitted_at DESC
`;

db.query(sql, (err, results) => {
    if(err) return res.status(500).json({
        message: 'Server error',
        error: err.message
    });
    res.json(results);
});

};

const getUploadSubjects = (req, res) => {
    const sql = `
        SELECT DISTINCT s.id, s.name
        FROM uploads u
        JOIN subjects s ON u.subject_id = s.id
        JOIN school_years sy ON u.school_year_id = sy.id
        WHERE sy.is_active = 1
        ORDER BY s.name ASC
    `;
    db.query(sql, (err, results) => {
        if(err) return res.status(500).json({ message: 'Server
error' });
        res.json(results);
    });
};

```

```

};

const getUploadSections = (req, res) => {

  const sql = `
    SELECT DISTINCT u.section
    FROM uploads u
    JOIN school_years sy ON u.school_year_id = sy.id
    WHERE sy.is_active = 1
    ORDER BY u.section ASC
  `;

  db.query(sql, (err, results) => {
    if(err) return res.status(500).json({ message: 'Server
error' });
    res.json(results.map(r => r.section));
  });

};

// =====
// GET UPLOAD COUNTS
// for admin dashboard cards
// =====

const getUploadCounts = (req, res) => {

  const sql = `
    SELECT
      COUNT(*)                                AS
total,
      SUM(CASE WHEN u.status = 'pending' THEN 1 ELSE 0 END)
AS pending,
      SUM(CASE WHEN u.status = 'approved' THEN 1 ELSE 0 END)
AS approved,
      SUM(CASE WHEN u.status = 'rejected' THEN 1 ELSE 0 END)
AS rejected,
      SUM(CASE WHEN u.status = 'locked' THEN 1 ELSE 0 END)
AS locked
    FROM uploads u
    JOIN school_years sy ON u.school_year_id = sy.id
    WHERE sy.is_active = 1
  `;

  db.query(sql, (err, results) => {
    if(err) return res.status(500).json({ message: 'Server
error' });
    res.json(results[0]);
  });
};

```

```

    });

};

// =====
// GET SINGLE UPLOAD + GRADES
// for admin review modal
// =====

const getUploadDetails = (req, res) => {

  const { id } = req.params;

  // first get the upload record
  const uploadSql = `
    SELECT
      u.id,
      u.grade_level,
      u.section,
      u.quarter,
      u.status,
      u.remarks,
      u.file_path,
      u.submitted_at,
      u.reviewed_at,
      t.full_name      AS teacher_name,
      t.prc_id,
      s.name           AS subject_name,
      sy.label         AS school_year
    FROM uploads u
    JOIN teachers      t  ON u.teacher_id      = t.id
    JOIN subjects      s  ON u.subject_id      = s.id
    JOIN school_years  sy ON u.school_year_id = sy.id
    WHERE u.id = ?
  `;

  db.query(uploadSql, [id], (err, uploadResult) => {

    if(err){
      console.error('Upload query error:', err.message);
      return res.status(500).json({
        message: 'Server error fetching upload',
        error: err.message
      });
    }

    if(!uploadResult || uploadResult.length === 0){

```

```

        return res.status(404).json({ message: 'Upload not
found' });
    }

    const upload = uploadResult[0];

    // then get grades for this upload
    const gradesSql = `
        SELECT
            g.id,
            g.score,
            g.status,
            g.quarter,
            st.lrn,
            st.full_name AS student_name
        FROM grades g
        JOIN students st ON g.student_id = st.id
        WHERE g.upload_id = ?
        ORDER BY st.full_name ASC
    `;

    db.query(gradesSql, [id], (err2, grades) => {

        if(err2){
            console.error('Grades query error:', err2.message);
            return res.status(500).json({
                message: 'Server error fetching grades',
                error: err2.message
            });
        }

        // grades might be empty for orphaned uploads - still
        return upload info
        const gradeList = grades || [];

        res.json({
            upload: upload,
            grades: gradeList,
            total: gradeList.length,
            passing: gradeList.filter(g => parseFloat(g.score) >=
75).length,
            failing: gradeList.filter(g => parseFloat(g.score) <
75).length
        });
    });
};

```

```

// =====
// UNAPPROVE UPLOAD
// returns approved grades back
// to pending so admin can fix
// a mistake or wait for updates
// =====

const unapproveUpload = (req, res) => {

  const { id } = req.params;

  db.query(
    'SELECT status FROM uploads WHERE id = ?',
    [id],
    (err, results) => {
      if(err) return res.status(500).json({ message: 'Server
error' });
      if(results.length === 0){
        return res.status(404).json({ message: 'Upload not
found' });
      }

      const currentStatus = results[0].status;

      if(currentStatus === 'locked'){
        return res.status(400).json({
          message: 'Locked uploads cannot be changed.'
        });
      }

      if(currentStatus === 'pending'){
        return res.status(400).json({
          message: 'Upload is already pending.'
        });
      }

      // set grades back to pending
      db.query(
        `UPDATE grades SET status = 'pending' WHERE upload_id
= ?`,
        [id],
        (err2) => {
          if(err2) return res.status(500).json({ message:
'Server error' });

          // set upload back to pending and clear remarks
          db.query(
            `UPDATE uploads

```

```

        SET status      = 'pending',
            remarks      = NULL,
            reviewed_at  = NULL
        WHERE id = ?`,
        [id],
        (err3) => {
            if(err3) return res.status(500).json({ message:
'Server error' });
            res.json({
                message: 'Upload returned to pending.'
            });
        }
    );
}
);
}
);
};

// =====
// TEACHER UPLOAD GRADES
// reads excel file, validates,
// saves upload + grade records
// =====

const uploadGrades = (req, res) => {

    const { subject_id, grade_level, section, quarter } =
req.body;
    const teacherUserId = req.user.id;

    if(!subject_id || !grade_level || !section || !quarter){
        return res.status(400).json({
            message: 'Subject, grade level, section and quarter are
required'
        });
    }

    if(!req.file){
        return res.status(400).json({ message: 'Excel file is
required' });
    }

    // — read excel file
    let worksheet;
    try {
        const workbook = xlsx.readFile(req.file.path);
        const sheetName = workbook.SheetNames[0];

```

```

        worksheet          = workbook.Sheets[sheetName];
    } catch(e) {
        return res.status(400).json({
            message: 'Could not read Excel file. Use the downloaded
template.'
        });
    }

    // — find LRN and Final Grade columns by scanning header
row
    const wsRange = xlsx.utils.decode_range(worksheet['!ref'] ||
'A1');
    let headerRowIdx = -1;
    let lrnColIdx    = -1;
    let gradeColIdx  = -1;

    for(let r = 0; r <= Math.min(wsRange.e.r, 20); r++){
        for(let c = 0; c <= wsRange.e.c; c++){
            const ref  = xlsx.utils.encode_cell({ r, c });
            const cell = worksheet[ref];
            if(!cell) continue;
            const val = String(cell.v || '').trim().toUpperCase();
            if(val === 'LRN'){ headerRowIdx = r; lrnColIdx = c; }
            if(val === 'FINAL GRADE' || val === 'SCORE') gradeColIdx
= c;
        }
        if(headerRowIdx !== -1 && gradeColIdx !== -1) break;
    }

    if(headerRowIdx === -1 || lrnColIdx === -1){
        return res.status(400).json({
            message: 'Could not find LRN column. Use the downloaded
template.'
        });
    }

    if(gradeColIdx === -1){
        return res.status(400).json({
            message: 'Could not find Final Grade column. Fill in
grades before uploading.'
        });
    }

    // — extract student rows below the header
    const dataRows = [];
        // stop at row 109 (index 108) — row 111 is instructions
in col D
        const lastDataRow = Math.min(wsRange.e.r, 108);

```



```

    for(let r = headerRowIndex + 1; r <= lastDataRow; r++){
      const lrnCell = worksheet[xlsx.utils.encode_cell({ r, c:
lrnColIdx })];
      const gradeCell = worksheet[xlsx.utils.encode_cell({ r, c:
gradeColIdx })];
      const lrn = lrnCell ? String(lrnCell.v ||
'').trim() : '';
      const score = gradeCell ?
parseFloat(gradeCell.v) : NaN;
      if(!lrn) continue;
      dataRows.push({ lrn, score });
    }

    if(dataRows.length === 0){
      return res.status(400).json({
        message: 'No student data found in the file.'
      });
    }

    // — validate all scores before doing anything
    const scoreErrors = [];
    dataRows.forEach((row, i) => {
      if(isNaN(row.score) || row.score < 0 || row.score > 100){
        scoreErrors.push(
          `Row ${headerRowIndex + i + 2}: ` +
          `Invalid grade "${row.score}" for LRN ${row.lrn}`
        );
      }
    });

    if(scoreErrors.length > 0){
      return res.status(400).json({
        message: 'Validation errors found in file',
        errors: scoreErrors
      });
    }

    // — main async flow
    const runUpload = async () => {

      // get active school year
      const [years] = await db.promise().query(
        'SELECT id FROM school_years WHERE is_active = 1 LIMIT
1'
      );
      if(years.length === 0){
        throw new Error('No active school year found. Set one in
Settings.');
```

```

    }
    const schoolYearId = years[0].id;

    // get teacher db record from user id
    const [teachers] = await db.promise().query(
        'SELECT id FROM teachers WHERE user_id = ?',
        [teacherUserId]
    );
    if(teachers.length === 0){
        throw new Error('Teacher record not found.');
```

```
    }
    const dbTeacherId = teachers[0].id;
```

```
    // — check for existing upload for same batch
```

```
    const [existing] = await db.promise().query(
        `SELECT id, status
        FROM uploads
        WHERE teacher_id      = ?
           AND subject_id     = ?
           AND grade_level    = ?
           AND section         = ?
           AND quarter         = ?
           AND school_year_id = ?`,
        [dbTeacherId, subject_id, grade_level, section, quarter,
        schoolYearId]
    );
```

```
    let uploadId      = null;
    let resubmitted = false;
```

```
    if(existing.length > 0){
```

```
        const existingStatus = existing[0].status;
        const existingId      = existing[0].id;
```

```
        // block if approved or locked
```

```
        if(existingStatus === 'approved'){
            throw new Error(
                'Grades for this subject and quarter are already
approved. ' +
                'Contact the admin if corrections are needed.'
            );
        }
    }
```

```
        if(existingStatus === 'locked'){
```

```
            throw new Error(
                'Grades for this subject and quarter are locked and
finalized. ' +

```

```

        'No further changes are allowed.'
    );
}

if(existingStatus === 'pending'){
    throw new Error(
        'A grade upload is already pending review. ' +
        'Wait for the admin to approve or reject it before
resubmitting.'
    );
}

// — RESUBMIT — status is 'rejected'
// delete old grades and update the upload record
await db.promise().query(
    'DELETE FROM grades WHERE upload_id = ?',
    [existingId]
);

// fix: use ? for every value — no hardcoded NULL
await db.promise().query(
    `UPDATE uploads
    SET file_path      = ?,
        status         = ?,
        remarks        = ?,
        reviewed_at    = ?,
        submitted_at   = NOW()
    WHERE id = ?`,
    [req.file.filename, 'pending', null, null, existingId]
);

uploadId      = existingId;
resubmitted   = true;

} else {

    // — NEW UPLOAD — create upload record
    const [uploadResult] = await db.promise().query(
        `INSERT INTO uploads
        (teacher_id, subject_id, school_year_id,
        grade_level, section, quarter, file_path, status)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)`,
        [
            dbTeacherId,
            subject_id,
            schoolYearId,
            grade_level,
            section,
            quarter,
            req.file.filename,
            'pending'
        ]
    );
}

```

```

        quarter,
        req.file.filename,
        'pending'
    ]
    );

    uploadId = uploadResult.insertId;
}

// — match LRNs to students and build grade inserts
const gradeInserts = [];
const notFound      = [];

for(const row of dataRows){
    const [studentResult] = await db.promise().query(
        'SELECT id FROM students WHERE lrn = ?',
        [row.lrn]
    );
    if(studentResult.length === 0){
        notFound.push(row.lrn);
    } else {
        gradeInserts.push([
            studentResult[0].id,
            subject_id,
            dbTeacherId,
            uploadId,
            schoolYearId,
            quarter,
            row.score,
            'pending'
        ]);
    }
}

if(gradeInserts.length === 0){
    // clean up the upload record if no students matched
    if(!resubmitted){
        await db.promise().query(
            'DELETE FROM uploads WHERE id = ?',
            [uploadId]
        );
    }
    throw new Error(
        `No matching students found in the system. ` +
        `LRNs not found: ${notFound.join(', ')} `
    );
}

```

```

// — bulk insert all grade records
await db.promise().query(
  `INSERT INTO grades
    (student_id, subject_id, teacher_id, upload_id,
     school_year_id, quarter, score, status)
  VALUES ?`,
  [gradeInserts]
);

return {
  uploadId,
  totalStudents: gradeInserts.length,
  notFound,
  resubmitted
};
};

// — run and respond
runUpload()
  .then(result => {

    const message = result.resubmitted
      ? `Grades updated and resubmitted successfully. ` +
        `${result.totalStudents} student(s) sent for admin
review.`
      : `Grades uploaded successfully. ` +
        `${result.totalStudents} student(s) submitted for
admin approval.`;

    res.status(201).json({
      message,
      upload_id:      result.uploadId,
      total_students: result.totalStudents,
      not_found:      result.notFound,
      resubmitted:    result.resubmitted
    });
  })
  .catch(err => {
    console.error('Upload grades error:', err.message);

    const clientErrors = [
      'already approved',
      'locked and finalized',
      'already pending',
      'No matching students',
      'No active school year',
      'Teacher record not found',
      'No student data',
    ]

```

```

        'Validation errors'
    ];

    const isClientError = clientErrors.some(e =>
        err.message.includes(e)
    );

    res.status(isClientError ? 400 : 500).json({
        message: err.message || 'Server error saving grades'
    });
});

};

// =====
// ADMIN APPROVE UPLOAD
// sets upload + all grades
// to approved – visible to students
// =====

const approveUpload = (req, res) => {

    const { id } = req.params;

    // approve all grades in this upload
    db.query(
        `UPDATE grades SET status = 'approved' WHERE upload_id =
        ?`,
        [id],
        (err) => {
            if(err) return res.status(500).json({ message: 'Server
            error' });

            // approve the upload batch
            db.query(
                `UPDATE uploads
                SET status = 'approved', reviewed_at = NOW()
                WHERE id = ?`,
                [id],
                (err2, result) => {
                    if(err2) return res.status(500).json({ message:
                    'Server error' });
                    if(result.affectedRows === 0){
                        return res.status(404).json({ message: 'Upload not
                        found' });
                    }
                    res.json({ message: 'Upload approved. Grades are now
                    visible to students.' });
                }
            );
        }
    );
};

```

```

        }
      );
    }
  );

};

// =====
// ADMIN REJECT UPLOAD
// sets upload to rejected
// teacher can see remarks
// and resubmit
// =====

const rejectUpload = (req, res) => {

  const { id } = req.params;
  const { remarks } = req.body;

  if(!remarks){
    return res.status(400).json({
      message: 'Please provide a reason for rejection'
    });
  }

  db.query(
    `UPDATE uploads
      SET status = 'rejected',
          remarks = ?,
          reviewed_at = NOW()
      WHERE id = ?`,
    [remarks, id],
    (err, result) => {
      if(err) return res.status(500).json({ message: 'Server
error' });
      if(result.affectedRows === 0){
        return res.status(404).json({ message: 'Upload not
found' });
      }
      res.json({ message: 'Upload rejected. Teacher will be
notified.' });
    }
  );

};

// =====
// ADMIN LOCK UPLOAD

```

```

// finalizes grades permanently
// cannot be edited after lock
// =====

const lockUpload = (req, res) => {

  const { id } = req.params;

  db.query(
    `UPDATE grades SET status = 'locked' WHERE upload_id = ?`,
    [id],
    (err) => {
      if(err) return res.status(500).json({ message: 'Server
error' });

      db.query(
        `UPDATE uploads SET status = 'locked' WHERE id = ?`,
        [id],
        (err2, result) => {
          if(err2) return res.status(500).json({ message:
'Server error' });
          if(result.affectedRows === 0){
            return res.status(404).json({ message: 'Upload not
found' });
          }
          res.json({ message: 'Upload locked. Grades are now
finalized.' });
        }
      );
    }
  );

};

// =====
// GET TEACHER OWN UPLOADS
// for teacher upload history
// =====

const getTeacherUploads = (req, res) => {

  const teacherUserId = req.user.id;

  const sql = `
    SELECT
      u.id,
      u.grade_level,
      u.section,

```



```

        u.quarter,
        u.status,
        u.remarks,
        u.submitted_at,
        u.reviewed_at,
        s.name AS subject_name,
        sy.label AS school_year
    FROM uploads u
    JOIN teachers t ON u.teacher_id = t.id
    JOIN subjects s ON u.subject_id = s.id
    JOIN school_years sy ON u.school_year_id = sy.id
    WHERE t.user_id = ?
        AND sy.is_active = 1
    ORDER BY u.submitted_at DESC
`;

    db.query(sql, [teacherUserId], (err, results) => {
        if(err) return res.status(500).json({ message: 'Server
error' });
        res.json(results);
    });

};

// =====
// GET STUDENT GRADES
// only approved grades visible
// filtered by school year
// and quarter
// =====

const getStudentGrades = (req, res) => {

    const userId = req.user.id;

    const sql = `
        SELECT
            g.score,
            g.quarter,
            g.status,
            s.name          AS subject_name,
            sy.label        AS school_year,
            sy.id           AS school_year_id,
            sy.is_active,
            u.grade_level,
            u.section
        FROM grades g
        JOIN subjects s ON g.subject_id = s.id
    `;

```

```

        JOIN uploads      u  ON g.upload_id      = u.id
        JOIN school_years sy ON g.school_year_id = sy.id
        JOIN students     st ON g.student_id     = st.id
        WHERE st.user_id = ?
              AND g.status IN ('approved', 'locked')
        ORDER BY sy.id DESC, s.name ASC, g.quarter ASC
    `;

    db.query(sql, [userId], (err, results) => {
        if(err) return res.status(500).json({ message: 'Server
error' });
        res.json(results);
    });

};

// =====
// GENERATE GRADE TEMPLATE
// reads base xlsx from assets
// fills header + student list
// sends as download
// =====

const generateGradeTemplate = async (req, res) => {

    const { subject_id, grade_level, section, quarter } =
req.query;

    if(!subject_id || !grade_level || !section || !quarter){
        return res.status(400).json({
            message: 'subject_id, grade_level, section and quarter
are required'
        });
    }

    try {

        // get subject name
        const [subjects] = await db.promise().query(
            'SELECT name FROM subjects WHERE id = ?',
            [subject_id]
        );

        if(subjects.length === 0){
            return res.status(404).json({ message: 'Subject not
found' });
        }
    }

```

```

const subjectName = subjects[0].name;

// get active school year
const [years] = await db.promise().query(
  'SELECT label FROM school_years WHERE is_active = 1
LIMIT 1'
);

const schoolYear = years.length > 0 ? years[0].label : '-';

// get students enrolled in this class
const [students] = await db.promise().query(
  `SELECT st.full_name, st.lrn
  FROM students st
  JOIN student_enrollments se ON st.id = se.student_id
  JOIN school_years sy ON se.school_year_id = sy.id
  WHERE se.grade_level = ?
  AND se.section = ?
  AND sy.is_active = 1
  AND st.is_archived = 0
  ORDER BY st.full_name ASC`,
  [grade_level, section]
);

// — build worksheet using array-of-arrays
// this gives full control over layout
// xlsx npm package supports cell styles via the xlsx-
style fork
// but since we only have xlsx, we use cell comments and
// width/height for structure, and rely on the clean
layout

const wb = xlsx.utils.book_new();

// build data rows
const wsData = [];

// Row 1: Title
wsData.push(['GRADE UPLOAD TEMPLATE', '', '']);

// Row 2: spacer
wsData.push(['', '', '']);

// Rows 3-7: header info
wsData.push(['Subject:', subjectName, 'Do not
change']);

```

```

        wsData.push(['School Year:', schoolYear, 'Do not
change']);
        wsData.push(['Grade Level:', grade_level, 'Do not
change']);
        wsData.push(['Section:', section, 'Do not
change']);
        wsData.push(['Quarter:', quarter + ' Quarter', 'Do not
change']);

// Row 8: spacer
wsData.push(['', '', '']);

// Row 9: column headers
wsData.push(['Full Name', 'LRN', 'Final Grade']);

// Rows 10-109: student data (100 rows)
for(let i = 0; i < 100; i++){
    if(i < students.length){
        wsData.push([students[i].full_name,
String(students[i].lrn), '']);
    } else {
        wsData.push(['', '', '']);
    }
}

// convert to worksheet
const ws = xlsx.utils.aoa_to_sheet(wsData);

// set column widths
ws['!cols'] = [
    { wch: 32 }, // A - Full Name
    { wch: 20 }, // B - LRN
    { wch: 16 } // C - Final Grade
];

// set row heights using !rows
ws['!rows'] = [
    { hpt: 28 }, // row 1 - title
    { hpt: 6 }, // row 2 - spacer
    { hpt: 20 }, // row 3 - Subject
    { hpt: 20 }, // row 4 - School Year
    { hpt: 20 }, // row 5 - Grade Level
    { hpt: 20 }, // row 6 - Section
    { hpt: 20 }, // row 7 - Quarter
    { hpt: 8 }, // row 8 - spacer
    { hpt: 22 } // row 9 - headers
];

```

```

// add to workbook
xlsx.utils.book_append_sheet(wb, ws, 'Grade Upload');

// write to buffer
const buffer = xlsx.write(wb, {
  type: 'buffer',
  bookType: 'xlsx'
});

// build filename
const filename = [
  'grades',
  subjectName.replace(/\s+/g, '_'),
  grade_level.replace(/\s+/g, '_'),
  section.replace(/\s+/g, '_'),
  quarter.replace(/\s+/g, '_')
].join('_') + '.xlsx';

res.setHeader(
  'Content-Disposition',
  `attachment; filename="${filename}"`
);
res.setHeader(
  'Content-Type',
  'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
);

res.send(buffer);

} catch(err) {
  console.error('Generate template error:', err);
  res.status(500).json({
    message: 'Server error generating template',
    error: err.message
  });
}

};

// =====
// PREVIEW GRADES
// reads uploaded excel file
// finds the student table by
// locating the LRN header row
// validates without saving
// =====

```

```

const previewGrades = (req, res) => {

  if(!req.file){
    return res.status(400).json({ message: 'No file uploaded'
});
  }

  let workbook, ws;

  try {
    workbook = xlsx.readFile(req.file.path);
    const sheetName = workbook.SheetNames[0];
    ws = workbook.Sheets[sheetName];
  } catch(e) {
    return res.status(400).json({
      message: 'Could not read file. Make sure it is .xlsx or
.xlsx.'
    });
  }

  // — find the row that contains the LRN header
  // the template has headers at row 9 (0-indexed: row 8)
  // we scan from row 0 to row 20 to find it

  const range = xlsx.utils.decode_range(ws['!ref'] ||
'A1:C1');
  let headerRow = -1;

  for(let r = 0; r <= Math.min(range.e.r, 20); r++){
    // check column A and column B for LRN keyword
    for(let c = 0; c <= 2; c++){
      const ref = xlsx.utils.encode_cell({ r, c });
      const cell = ws[ref];
      if(cell && String(cell.v || '').trim().toUpperCase() ===
'LRN'){
        headerRow = r;
        break;
      }
    }
    if(headerRow !== -1) break;
  }

  if(headerRow === -1){
    return res.status(400).json({
      message:
        'Could not find the "LRN" column header. ' +
        'Make sure you are using the downloaded template
without modifying the headers.'
    });
  }
}

```

```

    });
}

// — find which column is LRN, Full Name, Final Grade
let colLRN = -1;
let colName = -1;
let colGrade = -1;

for(let c = 0; c <= range.e.c; c++){
    const ref = xlsx.utils.encode_cell({ r: headerRow, c });
    const cell = ws[ref];
    if(!cell) continue;
    const val = String(cell.v || '').trim().toUpperCase();
    if(val === 'LRN') colLRN = c;
    if(val === 'FULL NAME') colName = c;
    if(val === 'FINAL GRADE') colGrade = c;
    if(val === 'SCORE') colGrade = c; // fallback
}

if(colLRN === -1){
    return res.status(400).json({
        message: 'LRN column not found. Use the downloaded
template.'
    });
}

if(colGrade === -1){
    return res.status(400).json({
        message:
            'Final Grade column not found. ' +
            'Fill in the Final Grade column before uploading.'
    });
}

// — read student rows — everything below the header row
const result = [];
let rowNum = 0;

// stop at row 109 (index 108) — row 111 is instructions
in col D
const lastDataRow = Math.min(range.e.r, 108);
for(let r = headerRow + 1; r <= lastDataRow; r++){

    const lrnRef = xlsx.utils.encode_cell({ r, c: colLRN });
    const nameRef = colName !== -1
        ? xlsx.utils.encode_cell({ r, c: colName })
        : null;

```

```

    const gradeRef = xlsx.utils.encode_cell({ r, c: colGrade
});

    const lrnCell    = ws[lrnRef];
    const nameCell   = nameRef ? ws[nameRef] : null;
    const gradeCell  = ws[gradeRef];

    const lrn        = lrnCell    ? String(lrnCell.v    || '').trim()
: '';
    const name        = nameCell   ? String(nameCell.v   || '').trim()
: '';
    const raw         = gradeCell ? gradeCell.v : '';
    const score       = parseFloat(raw);

    // skip completely empty rows
    if(!lrn && !name && (raw === '' || raw === undefined)){
        continue;
    }

    rowNum++;

    if(!lrn){
        result.push({
            row:    r + 1,
            lrn:    '-',
            name:    name || '-',
            score:   '-',
            valid:   false,
            error:   'Missing LRN'
        });
        continue;
    }

    if(raw === '' || raw === undefined || raw === null){
        result.push({
            row:    r + 1,
            lrn,
            name,
            score:   '(empty)',
            valid:   false,
            error:   'Final Grade is empty - fill in the grade
before uploading'
        });
        continue;
    }

    if(isNaN(score)){
        result.push({

```



```

        row:    r + 1,
        lrn,
        name,
        score: raw,
        valid: false,
        error: ` "${raw}" is not a valid number `
    });
    continue;
}

if(score < 0 || score > 100){
    result.push({
        row:    r + 1,
        lrn,
        name,
        score,
        valid: false,
        error: `Grade ${score} is out of range (must be 0-
100)`
    });
    continue;
}

result.push({
    row:    r + 1,
    lrn,
    name,
    score,
    valid: true,
    error: null
});
}

if(result.length === 0){
    return res.status(400).json({
        message:
            'No student data found below the header row. ' +
            'Make sure students are listed below the Full Name |
LRN | Final Grade headers.'
    });
}

res.json({ rows: result });

};

// =====
// GET TEACHER'S GRADE RECORDS

```

```

// returns all grades submitted
// by this teacher - no admin needed
// =====

const getTeacherGradeRecords = (req, res) => {

  const teacherUserId = req.user.id;

  const sql = `
    SELECT
      g.id,
      g.score,
      g.quarter,
      g.status,
      st.full_name AS student_name,
      st.lrn,
      s.name AS subject_name,
      u.grade_level,
      u.section,
      sy.label AS school_year
    FROM grades g
    JOIN students st ON g.student_id = st.id
    JOIN subjects s ON g.subject_id = s.id
    JOIN uploads u ON g.upload_id = u.id
    JOIN teachers t ON g.teacher_id = t.id
    JOIN school_years sy ON g.school_year_id = sy.id
    WHERE t.user_id = ?
      AND sy.is_active = 1
    ORDER BY u.grade_level, u.section, st.full_name, s.name
  ASC
  `;

  db.query(sql, [teacherUserId], (err, results) => {
    if(err){
      return res.status(500).json({
        message: 'Server error',
        error: err.message
      });
    }
    res.json(results);
  });

};

const exportTeacherGrades = async (req, res) => {

  const teacherUserId = req.user.id;
  const { subject, section, quarter, status } = req.query;

```

```

let sql = `
  SELECT
    st.full_name AS student_name,
    st.lrn,
    s.name AS subject_name,
    u.grade_level,
    u.section,
    g.quarter,
    g.score,
    g.status,
    sy.label AS school_year
  FROM grades g
  JOIN students st ON g.student_id = st.id
  JOIN subjects s ON g.subject_id = s.id
  JOIN uploads u ON g.upload_id = u.id
  JOIN teachers t ON g.teacher_id = t.id
  JOIN school_years sy ON g.school_year_id = sy.id
  WHERE t.user_id = ?
  AND sy.is_active = 1
`;

const params = [teacherUserId];

if(subject){ sql += ' AND s.name = ' + subject + ' '; }
if(section){ sql += ' AND u.section = ' + section + ' '; }
if(quarter){ sql += ' AND g.quarter = ' + quarter + ' '; }
if(status){ sql += ' AND g.status = ' + status + ' '; }

sql += ' ORDER BY u.grade_level, u.section, st.full_name, s.name ASC';

try {

  const [records] = await db.promise().query(sql, params);

  if(records.length === 0){
    return res.status(404).json({
      message: 'No grade records found for the selected filters'
    });
  }

  // build excel

```

```

const wb = xlsx.utils.book_new();

// title row and headers
const wsData = [];
wsData.push(['GRADE RECORDS EXPORT']);
wsData.push(['School Year: ${records[0].school_year}']);
wsData.push([]);
wsData.push([
  'Student Name',
  'LRN',
  'Subject',
  'Grade Level',
  'Section',
  'Quarter',
  'Score',
  'Status'
]);

records.forEach(r => {
  wsData.push([
    r.student_name,
    String(r.lrn),
    r.subject_name,
    r.grade_level,
    r.section,
    r.quarter + ' Quarter',
    r.score,
    r.status.charAt(0).toUpperCase() + r.status.slice(1)
  ]);
});

const ws = xlsx.utils.aoa_to_sheet(wsData);

ws['!cols'] = [
  { wch: 28 },
  { wch: 18 },
  { wch: 20 },
  { wch: 14 },
  { wch: 12 },
  { wch: 12 },
  { wch: 10 },
  { wch: 12 }
];

xlsx.utils.book_append_sheet(wb, ws, 'Grade Records');

const buffer = xlsx.write(wb, { type: 'buffer', bookType:
'xlsx' });

```

```

        const filename = `grade_records_${Date.now()}.xlsx`;
        res.setHeader('Content-Disposition', `attachment;
filename="${filename}"`);
        res.setHeader('Content-Type',
            'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet'
        );
        res.send(buffer);

    } catch(err) {
        console.error('Export grades error:', err);
        res.status(500).json({ message: 'Server error exporting
grades' });
    }
};

module.exports = {
    upload,
    getAllUploads,
    getUploadCounts,
    getUploadDetails,
    uploadGrades,
    approveUpload,
    rejectUpload,
    lockUpload,
    unapproveUpload,
    getTeacherUploads,
    getTeacherGradeRecords,
    getStudentGrades,
    previewGrades,
    generateGradeTemplate,
    getUploadSubjects,
    getUploadSections,
    exportTeacherGrades
};

```

Frontend API Helper

```
const API_URL = 'https://grading-portal-system-
production.up.railway.app/api';

// get token from localStorage
function getToken() {
  return localStorage.getItem('token');
}
// reusable fetch function
async function apiRequest(endpoint, method = 'GET', body = null)
{
  const options = {
    method,
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${getToken()}`
    }
  };
  if(body) options.body = JSON.stringify(body);
  let response, data;
  try {
    response = await fetch(`${API_URL}${endpoint}`, options);
  } catch(err) {
    // network error – server is down or no connection
    // do NOT logout, just throw so caller handles it
    throw new Error('Cannot connect to server. Please check your
connection.');
```

```
  }
  try {
    data = await response.json();
  } catch(e) {
    data = {};
  }
  // 401 = token expired or invalid
  // only redirect if we actually have a token stored
  // this prevents redirect loops
  if(response.status === 401){
    const storedToken = localStorage.getItem('token');
    if(storedToken){
      // token exists but server rejected it – truly expired
      console.warn('Token expired – redirecting to login');
      localStorage.clear();
      // redirect to the right login page based on folder depth
      const isInSubfolder = window.location.pathname
        .split('/').length > 2;
      window.location.href = isInSubfolder
        ? '../index.html'
        : 'index.html';
    }
    return;
  }
  // 403 = forbidden but still logged in
  // do NOT logout – just throw so caller handles it
```

```
if(response.status === 403){
  throw new Error(data.message || 'Access denied');
}
if(response.status === 404){
  throw new Error(data.message || 'Not found'); }
if(!response.ok){
  throw new Error(data.message || `Request failed
(${response.status})`); }
return data;}
```

Student Routes

```
const express = require('express');
const router = express.Router();

const {
  getAllStudents,
  getArchivedStudents,
  getStudent,
  getMyProfile,
  getStudentsByClass,
  addStudent,
  updateStudent,
  deleteStudent,
  archiveStudent,
  bulkArchiveStudents,
  bulkRestoreStudents,
  bulkDeleteStudents
} = require('../controllers/studentController');
const { importUpload, importStudentsFromExcel } =
  require('../controllers/studentImportController');
const { verifyToken, isAdmin } =
  require('../middleware/authMiddleware');

// — SPECIFIC NAMED ROUTES FIRST
// these must come before /:id or Express will
// treat 'archived', 'by-class', 'my-profile'
// as ID parameters and route to getStudent
router.get( '/archived', verifyToken, isAdmin, getArchivedStudents);
router.get( '/my-profile', verifyToken, getMyProfile);

// accessible by any logged-in user (teacher or admin)
// no isAdmin – teachers need this to view their class lists
router.get( '/by-class', verifyToken, getStudentsByClass);

// — BULK ACTIONS
router.post( '/bulk/archive', verifyToken, isAdmin, bulkArchiveStudents);
router.post( '/bulk/restore', verifyToken, isAdmin, bulkRestoreStudents);
router.post( '/bulk/delete', verifyToken, isAdmin, bulkDeleteStudents);

// — EXCEL IMPORT
router.post( '/import', verifyToken, isAdmin, importUpload.single(
  'studentFile'), importStudentsFromExcel);

// — GENERAL CRUD
// /:id routes LAST so named routes above are matched first
router.get( '/', verifyToken, isAdmin, getAllStudents);
```



```
router.get(('/:id', verifyToken, isAdmin, getStudent);
router.post( '/', verifyToken, isAdmin, addStudent);
router.put(('/:id', verifyToken, isAdmin, updateStudent);
router.delete(('/:id', verifyToken, isAdmin, deleteStudent);
router.put(('/:id/archive', verifyToken, isAdmin, archiveStudent)
;

module.exports = router;
```

Grade Routes

```
const express = require('express');
const router = express.Router();

const {
  upload,
  getAllUploads,
  getUploadCounts,
  getUploadDetails,
  getUploadSubjects,
  getUploadSections,
  uploadGrades,
  approveUpload,
  rejectUpload,
  lockUpload,
  unapproveUpload,
  getTeacherUploads,
  getTeacherGradeRecords,
  getStudentGrades,
  previewGrades,
  generateGradeTemplate,
  exportTeacherGrades
} = require('../controllers/gradeController');

const {
  verifyToken,
  isAdmin,
  isTeacher,
  isStudent,
  isTeacherOrAdmin
} = require('../middleware/authMiddleware');

// — TEACHER ROUTES
router.get('/my-uploads', verifyToken, isTeacherOrAdmin,
  getTeacherUploads);
router.get('/my-grade-records', verifyToken, isTeacherOrAdmin,
  getTeacherGradeRecords);
router.get('/my-grades', verifyToken,
  isStudent, getStudentGrades);
router.get('/template', verifyToken, isTeacherOrAdmin,
  generateGradeTemplate);
router.get('/export', verifyToken, isTeacherOrAdmin,
  exportTeacherGrades);

router.post('/preview', verifyToken, isTeacherOrAdmin,
  upload.single('gradeFile'), previewGrades);
router.post('/upload', verifyToken, isTeacherOrAdmin,
  upload.single('gradeFile'), uploadGrades);
```

```

// — ADMIN SPECIFIC NAMED ROUTES — must come before /uploads/:id
router.get('/uploads/counts', verifyToken, isAdmin,
getUploadCounts);
router.get('/uploads/subjects', verifyToken, isAdmin,
getUploadSubjects);
router.get('/uploads/sections', verifyToken, isAdmin,
getUploadSections);

// — ADMIN UPLOAD MANAGEMENT
router.get('/uploads', verifyToken, isAdmin,
getAllUploads);
router.get('/uploads/:id', verifyToken, isAdmin,
getUploadDetails);
router.put('/uploads/:id/approve', verifyToken, isAdmin,
approveUpload);
router.put('/uploads/:id/reject', verifyToken, isAdmin,
rejectUpload);
router.put('/uploads/:id/lock', verifyToken, isAdmin,
lockUpload);
router.put('/uploads/:id/unapprove', verifyToken, isAdmin,
unapproveUpload);

module.exports = router;

```

Report Controller

```
const db = require('../config/db');

// =====
// STUDENT REPORT
// shows all students with
// enrollment info per school year
// =====

const getStudentReport = (req, res) => {

  const { school_year_id, grade_level, section } = req.query;

  let sql = `
    SELECT
      s.id,
      s.lrn,
      s.full_name,
      se.grade_level,
      se.section,
      se.adviser,
      sy.label      AS school_year,
      u.status      AS account_status,
      s.is_archived
    FROM students s
    LEFT JOIN users u ON s.user_id = u.id
    LEFT JOIN student_enrollments se ON s.id = se.student_id
    LEFT JOIN school_years sy ON se.school_year_id = sy.id
    WHERE 1=1
  `;

  const params = [];

  if(school_year_id){
    sql += ' AND sy.id = ?';
    params.push(school_year_id);
  }

  if(grade_level){
    sql += ' AND se.grade_level = ?';
    params.push(grade_level);
  }

  if(section){
    sql += ' AND se.section = ?';
    params.push(section);
  }
}
```

```

    sql += ' ORDER BY sy.label DESC, se.grade_level, se.section,
s.full_name ASC';

    db.query(sql, params, (err, results) => {
        if(err) return res.status(500).json({ message: 'Server
error', error: err.message });
        res.json(results);
    });

};

// =====
// TEACHER REPORT
// shows all teachers with
// their assignments
// =====

const getTeacherReport = (req, res) => {

    const { school_year_id } = req.query;

    let sql = `
        SELECT
            t.id,
            t.prc_id,
            t.full_name,
            u.status      AS account_status,
            t.is_archived,
            ta.grade_level,
            ta.section,
            s.name        AS subject_name,
            sy.label      AS school_year
        FROM teachers t
        LEFT JOIN users u ON t.user_id = u.id
        LEFT JOIN teacher_assignments ta ON t.id = ta.teacher_id
        LEFT JOIN subjects s ON ta.subject_id = s.id
        LEFT JOIN school_years sy ON ta.school_year_id = sy.id
        WHERE 1=1
    `;

    const params = [];

    if(school_year_id){
        sql += ' AND (sy.id = ? OR sy.id IS NULL)';
        params.push(school_year_id);
    }

    sql += ' ORDER BY t.full_name ASC, sy.label DESC';

```

```

    db.query(sql, params, (err, results) => {
      if(err) return res.status(500).json({ message: 'Server
error', error: err.message });
      res.json(results);
    });

  };

// =====
// GRADE UPLOAD REPORT
// shows all uploads with
// approval status summary
// =====

const getGradeUploadReport = (req, res) => {

  const { school_year_id, quarter, status } = req.query;

  let sql = `
    SELECT
      u.id,
      u.grade_level,
      u.section,
      u.quarter,
      u.status,
      u.submitted_at,
      u.reviewed_at,
      t.full_name AS teacher_name,
      t.prc_id,
      s.name AS subject_name,
      sy.label AS school_year,
      (
        SELECT COUNT(*)
        FROM grades g
        WHERE g.upload_id = u.id
      ) AS total_students,
      (
        SELECT COUNT(*)
        FROM grades g
        WHERE g.upload_id = u.id
          AND g.score >= 75
      ) AS passing,
      (
        SELECT COUNT(*)
        FROM grades g
        WHERE g.upload_id = u.id
          AND g.score < 75

```

```

        ) AS failing
    FROM uploads u
    JOIN teachers      t  ON u.teacher_id      = t.id
    JOIN subjects      s  ON u.subject_id      = s.id
    JOIN school_years sy ON u.school_year_id = sy.id
    WHERE 1=1
  `;

const params = [];

if(school_year_id){
  sql += ' AND sy.id = ?';
  params.push(school_year_id);
}

if(quarter){
  sql += ' AND u.quarter = ?';
  params.push(quarter);
}

if(status){
  sql += ' AND u.status = ?';
  params.push(status);
}

sql += ' ORDER BY sy.label DESC, u.submitted_at DESC';

db.query(sql, params, (err, results) => {
  if(err) return res.status(500).json({
    message: 'Server error',
    error: err.message
  });
  res.json(results);
});

};

// =====
// SCHOOL YEAR HISTORY
// shows summary per school year
// =====

const getSchoolYearHistory = (req, res) => {

  const sql = `
    SELECT
      sy.id,
      sy.label,

```

```

sy.is_active,
sy.created_at,
qc.total_quarters,
qc.active_quarter,
(
    SELECT COUNT(DISTINCT se.student_id)
    FROM student_enrollments se
    WHERE se.school_year_id = sy.id
) AS total_students,
(
    SELECT COUNT(DISTINCT ta.teacher_id)
    FROM teacher_assignments ta
    WHERE ta.school_year_id = sy.id
) AS total_teachers,
(
    SELECT COUNT(*)
    FROM uploads u
    WHERE u.school_year_id = sy.id
) AS total_uploads,
(
    SELECT COUNT(*)
    FROM uploads u
    WHERE u.school_year_id = sy.id
        AND u.status = 'approved'
) AS approved_uploads,
(
    SELECT COUNT(*)
    FROM uploads u
    WHERE u.school_year_id = sy.id
        AND u.status = 'pending'
) AS pending_uploads,
(
    SELECT COUNT(*)
    FROM uploads u
    WHERE u.school_year_id = sy.id
        AND u.status = 'locked'
) AS locked_uploads
FROM school_years sy
LEFT JOIN quarters_config qc ON qc.school_year_id = sy.id
ORDER BY sy.created_at DESC
`;

```

```

db.query(sql, (err, results) => {
    if(err) return res.status(500).json({
        message: 'Server error',
        error: err.message
    });
    res.json(results);
}

```



```

    });

};

// =====
// REPORT SUMMARY COUNTS
// for the report section cards
// =====

const getReportSummary = (req, res) => {

  const sql = `
    SELECT
      (
        SELECT COUNT(DISTINCT se.student_id)
        FROM student_enrollments se
        JOIN school_years sy ON se.school_year_id = sy.id
        WHERE sy.is_active = 1
      ) AS total_students,
      (
        SELECT COUNT(DISTINCT ta.teacher_id)
        FROM teacher_assignments ta
        JOIN school_years sy ON ta.school_year_id = sy.id
        WHERE sy.is_active = 1
      ) AS total_teachers,
      (
        SELECT COUNT(*)
        FROM uploads u
        JOIN school_years sy ON u.school_year_id = sy.id
        WHERE u.status = 'approved'
          AND sy.is_active = 1
      ) AS approved_uploads,
      (
        SELECT COUNT(*)
        FROM uploads u
        JOIN school_years sy ON u.school_year_id = sy.id
        WHERE u.status = 'pending'
          AND sy.is_active = 1
      ) AS pending_uploads,
      (
        SELECT label
        FROM school_years
        WHERE is_active = 1
        LIMIT 1
      ) AS active_school_year
  `;

  db.query(sql, (err, results) => {

```

```

        if(err) return res.status(500).json({
            message: 'Server error',
            error: err.message
        });
        res.json(results[0]);
    });

};

// =====
// for the archive
// =====

// get full record of a specific school year
const getSchoolYearRecord = (req, res) => {

    const { year_id } = req.params;
    const { type } = req.query;
    // type = students | teachers | grades | uploads

    if(type === 'students'){

        const sql = `
            SELECT
                s.lrn,
                s.full_name,
                se.grade_level,
                se.section,
                se.adviser,
                COALESCE(u.status, 'not_activated') AS account_status
            FROM students s
            JOIN student_enrollments se ON s.id = se.student_id
            LEFT JOIN users u ON s.user_id = u.id
            WHERE se.school_year_id = ?
            ORDER BY se.grade_level, se.section, s.full_name ASC
        `;

        db.query(sql, [year_id], (err, results) => {
            if(err) return res.status(500).json({ message: 'Server
error' });
            res.json(results);
        });

    } else if(type === 'teachers'){

        const sql = `
            SELECT DISTINCT
                t.prc_id,

```

```

        t.full_name,
        s.name          AS subject_name,
        ta.grade_level,
        ta.section,
        COALESCE(u.status, 'not_activated') AS account_status
FROM teachers t
JOIN teacher_assignments ta ON t.id = ta.teacher_id
JOIN subjects s ON ta.subject_id = s.id
LEFT JOIN users u ON t.user_id = u.id
WHERE ta.school_year_id = ?
ORDER BY t.full_name, s.name ASC
`;

db.query(sql, [year_id], (err, results) => {
  if(err) return res.status(500).json({ message: 'Server
error' });
  res.json(results);
});
} else if(type === 'uploads'){

const sql = `
SELECT
  u.id,
  u.quarter,
  u.grade_level,
  u.section,
  u.status,
  u.submitted_at,
  t.full_name  AS teacher_name,
  s.name       AS subject_name,
  COUNT(g.id)  AS total_students,
  SUM(CASE WHEN g.score >= 75 THEN 1 ELSE 0 END) AS
passing,
  SUM(CASE WHEN g.score < 75 THEN 1 ELSE 0 END) AS failing
FROM uploads u
JOIN teachers t ON u.teacher_id = t.id
JOIN subjects s ON u.subject_id = s.id
LEFT JOIN grades g ON g.upload_id = u.id
WHERE u.school_year_id = ?
GROUP BY u.id, u.quarter, u.grade_level, u.section,
         u.status, u.submitted_at,
         t.full_name, s.name
ORDER BY u.submitted_at DESC
`;

db.query(sql, [year_id], (err, results) => {
  if(err) return res.status(500).json({ message: 'Server
error' });

```

```

        res.json(results);
    });

} else if(type === 'grades'){
    const sql = `
        SELECT
            st.lrn,
            st.full_name AS student_name,
            se.grade_level,
            se.section,
            s.name AS subject_name,
            g.quarter,
            g.score,
            g.status
        FROM grades g
        JOIN students st ON g.student_id = st.id
        JOIN subjects s ON g.subject_id = s.id
        JOIN student_enrollments se
            ON se.student_id = st.id
            AND se.school_year_id = ?
        WHERE g.school_year_id = ?
        ORDER BY se.grade_level, se.section, st.full_name, s.name
    ASC
    `;

    db.query(sql, [year_id, year_id], (err, results) => {
        if(err) return res.status(500).json({ message: 'Server
error' });
        res.json(results);
    });
    //summary query

} else {

    const sql = `
        SELECT
            sy.id,
            sy.label,
            sy.is_active,
            sy.created_at,
            (
                SELECT COUNT(DISTINCT se.student_id)
                FROM student_enrollments se
                WHERE se.school_year_id = sy.id
            ) AS total_students,
            (
                SELECT COUNT(DISTINCT ta.teacher_id)
                FROM teacher_assignments ta
    
```

```

        WHERE ta.school_year_id = sy.id
    ) AS total_teachers,
    (
        SELECT COUNT(*)
        FROM uploads u
        WHERE u.school_year_id = sy.id
    ) AS total_uploads,
    (
        SELECT COUNT(*)
        FROM uploads u
        WHERE u.school_year_id = sy.id
            AND u.status = 'approved'
    ) AS approved,
    (
        SELECT COUNT(*)
        FROM uploads u
        WHERE u.school_year_id = sy.id
            AND u.status = 'locked'
    ) AS locked
FROM school_years sy
WHERE sy.id = ?
`;

db.query(sql, [year_id], (err, results) => {
    if(err) return res.status(500).json({
        message: 'Server error'
    });
    if(results.length === 0){
        return res.status(404).json({
            message: 'School year not found'
        });
    }
    res.json(results[0]);
});

}

};

module.exports = {
    getStudentReport,
    getTeacherReport,
    getGradeUploadReport,
    getSchoolYearHistory,
    getReportSummary,
    getSchoolYearRecord
};

```